Arquitectura hexagonal de IMSI — guía de estudio

Documento personal, no entregable. Explica qué es el estilo, por qué se eligió en este proyecto, qué muestra exactamente el diagrama entregado, y cómo se ve el sistema terminado en forma de flujos con pseudocódigo.

Fuentes usadas: `Design.pdf` (p. 24), `Architecture_Diagram_Testigos_De_Meluk.drawio` (versión del 10-nov), `Arquitectura_Modular_Hexagonal.drawio` del workspace, y los requisitos del análisis.

---

1. Resumen en diez líneas

1. La arquitectura del sistema es **hexagonal**: puertos y adaptadores alrededor de un dominio aislado. Esa es la respuesta que se da si preguntan por el estilo, y es la que está en la entrega.
2. El sistema vive en **un solo despliegue** (Django + Celery), no en microservicios. Eso es despliegue, no estilo; solo se menciona si la pregunta es "¿cómo se despliega?".
3. El **dominio** no sabe que existe HTTP, PostgreSQL, DeepSeek ni Siigo.
4. Lo que el mundo exterior puede pedirle al sistema se declara como **puerto de entrada** (interfaz).
5. Lo que el sistema necesita del exterior se declara como **puerto de salida** (interfaz).
6. Los **adaptadores** son las implementaciones concretas: controladores Django, repositorios Postgres, notificadores SMTP.
7. La dirección de las dependencias va **siempre hacia el dominio**. Nunca al revés.
8. Cambiar de base de datos, de proveedor de IA o de canal de entrada no toca el dominio: se sustituye el adaptador.
9. Los **DTO** cruzan la frontera; las entidades no se serializan ni se devuelven tal cual.
10. El diagrama entregado dibuja un hexágono (el núcleo IMSI); el macro de trabajo lo forman varios módulos así, todos dentro del mismo despliegue.

---

2. Qué estilo es y cómo se llama

La respuesta al estilo es **hexagonal** (puertos y adaptadores). Punto.

"Monolito modular" **no** es un segundo estilo que competir con el hexagonal: describe cómo se **despliega** el sistema —un proceso Django, varios módulos dentro, una base de datos— por oposición a microservicios. Si te preguntan por la arquitectura, decir "monolito modular" primero suena a que no contestaste lo que te preguntaron.

Regla práctica:

Pregunta	Respuesta
¿Qué arquitectura usaron?	Hexagonal (puertos y adaptadores)
¿Cómo se organiza por dentro?	Dominio al centro; adaptadores en el borde; dependencias hacia el dominio

Pregunta	Respuesta
¿Es microservicios? / ¿cómo lo despliegan?	Un solo despliegue Django con módulos internos

Y lo que sí es una composición de patrones es **esto**, en el orden en que se nombra:

Capa de la respuesta	Nombre correcto	Qué aporta aquí
Arquitectura de despliegue	**Monolito modular**	un proceso Django, módulos con bordes; no microservicios
Arquitectura de módulo	**Hexagonal / puertos y adaptadores** (Alistair Cockburn)	el dominio queda aislado; entradas y salidas son interfaces
Organización interna	**Capas concéntricas** (aplicación → dominio)	separa casos de uso de reglas de negocio
Patrón de acceso a datos	**Repository + Mapper / Data Mapper**	Postgres no aparece en el dominio
Patrón de comunicación	**DTO** en la frontera	el adaptador no filtra el modelo interno
Patrones exigidos por NFR-05-01	**Strategy, Factory, Observer, Singleton**	se implementan dentro de este esquema (ver §7)

Lo que **no** es:

- No es MVC clásico: no hay modelo-activo con lógica en el ORM.
- No es arquitectura en 3 capas (presentación/negocio/datos): esa deja el ORM en el centro.
- No es microservicios: hay un despliegue.
- No es "carpetas bonitas": si el dominio importa `django.db`, no es hexagonal aunque el diagrama lo diga.

2.1 Estilo, patrón, paradigma y metodología: no son sinónimos

Esta es la confusión más frecuente: se usan «DDD», «hexagonal», «monolito modular» y «cliente-servidor» como si compitieran. No compiten: cada uno responde a una pregunta distinta.

Nivel	La pregunta que responde	Ejemplos	En IMSI
Paradigma o enfoque de modelado	¿cómo pienso el negocio antes de programarlo?	DDD, modelado orientado a objetos	**DDD táctico**: agregados, entidades, value objects, repositorios, servicios de dominio, políticas
Estilo arquitectónico	¿cómo se organiza el sistema y cómo se relacionan sus partes?	cliente-servidor, capas, hexagonal, microservicios, EDA	**Hexagonal** para las dependencias; **cliente-servidor** para la comunicación; **monolito modular** para el despliegue; eventos para la integración
Patrón arquitectónico	¿qué solución conocida aplico a este problema recurrente?	MVC, API Gateway, Repository, Unit of Work, CQRS	**Repository**, **Unit of Work**, **Observer**, **DTO** en la frontera
Patrón de diseño	¿qué solución aplico a un problema de código?	Strategy, Factory, Singleton	los que exige NFR-05-01

DDD no es un estilo arquitectónico. Domain-Driven Design dice *cómo modelar el negocio*: qué es un agregado, dónde están las invariantes, qué lenguaje usa el equipo, qué límites tiene cada contexto. No dice si

el despliegue es uno o cinco, ni en qué carpeta va cada clase. Un dominio modelado con DDD puede vivir en capas, en hexagonal o dentro de microservicios.

Hexagonal sí es un estilo. Dice cómo se organizan las dependencias: el dominio al centro, y todo lo demás (HTTP, base de datos, IA, Siigo, USB) como adaptadores conectados por puertos. No dice nada del modelo del negocio; ese trabajo lo hace DDD.

Por eso la combinación es la habitual: DDD produce un dominio rico y hexagonal lo protege. El material del curso los presenta juntos por la misma razón: la lámina «Hexagonal Architecture — Ports & Adapters» lleva como etiquetas *Domain isolation · Ports & adapters · DDD* y cita el repositorio `domain-driven-hexagon`, que es exactamente esa combinación. Que el hexagonal esté en la lámina de DDD no lo convierte en DDD.

¿Puede haber varios estilos en un mismo sistema? Sí, porque cada uno responde a una dimensión distinta. En IMSI:

Pregunta	Respuesta	Dónde se ve en el proyecto
¿Cómo se comunican los clientes con el sistema?	**Cliente-servidor**	navegador ↔ nginx ↔ Django REST / WebSocket
¿Cuántos despliegues hay?	**Uno** (monolito modular)	un contenedor Django + workers Celery, una base
¿Cómo se organizan las dependencias?	**Hexagonal**	el diagrama de la entrega: puertos y adaptadores
¿Cómo se integra con otros sistemas?	**Por eventos internos y archivos planos**	`QuotationApproved`, exportación a Siigo, SMTP
¿Cómo se expone la API?	**REST** con JWT	DRF + SimpleJWT
¿Cómo se modela el negocio?	**DDD táctico**	agregados `Quotation`, `WorkOrder`, `BackupFile`; VOs `Money`, `PriceRange`; políticas

La frase que conviene memorizar, porque cubre todas las preguntas en un solo orden:

> *«El estilo arquitectónico es **hexagonal** (puertos y adaptadores); se despliega como **monolito modular**; la comunicación es **cliente-servidor**; el dominio se modeló con **DDD táctico**; y los patrones obligatorios son Strategy, Factory, Observer y Singleton.»*

Si te preguntan solo por el estilo, la respuesta es **hexagonal**. Si preguntan por el modelo del dominio, es **DDD**. Si preguntan por el despliegue, es **monolito modular**. Decir «somos DDD» como si fuera el estilo es lo que hace pensar que no hay respuesta.

2.2 Lo que el curso considera «cuándo usar hexagonal»

La lámina del curso da cuatro criterios. Los cuatro se cumplen en IMSI, y cada uno tiene evidencia en el análisis:

Criterio del curso	Evidencia en IMSI
Dominio complejo, con reglas que cambian	rangos de precio, inmutabilidad de la cotización aprobada, estados de la O.T., anticipo del 30 %, secuencia de numeración
Varias interfaces de entrada	REST, WebSocket del chatbot y cron de respaldos
Cambios frecuentes de base o tecnología	Siigo sin API, IA reemplazable, respaldo en USB o NAS
Testabilidad del dominio sin infraestructura	NFR-05-02: cobertura ≥ 70 % con datos sintéticos y sin base real

2.3 ¿Hexagonal es un patrón de diseño? ¿Y esto es orientado a servicios?

Hexagonal es un patrón arquitectónico, no un patrón de diseño. La diferencia está en el alcance, y se ve en una pregunta:

Nivel	Qué abarca	Ejemplos	Dónde se nota
Patrón de diseño	colaboración entre clases u objetos	Strategy, Factory, Observer, Singleton	en uno o dos archivos
Patrón arquitectónico (estilo)	organiza el sistema completo y fija quién puede depender de quién	capas, MVC, hexagonal, cliente-servidor, SOA, microservicios	en el árbol de carpetas, los imports y el despliegue

Hexagonal decide dónde vive el dominio y qué puede importar a qué: eso es más grande que una clase y menos que un lenguaje. Por eso es **arquitectónico**. Los cuatro patrones que exige NFR-05-01 (Strategy, Factory, Observer, Singleton) sí son de diseño, y viven dentro de las piezas que hexagonal separa.

¿Y orientado a servicios (SOA)? No. SOA es otro estilo, con marcas muy concretas:

- las funcionalidades se exponen como **servicios con contrato versionado**;
- los servicios se invocan **por red** (SOAP, REST o bus de mensajes);
- hay registro o bus de servicios, y cada servicio puede desplegarse por separado.

En IMSI ninguna de las tres se cumple:

Marca de SOA	En IMSI
Contratos entre servicios por red	los módulos se llaman **en el mismo proceso**, por su `api.py`
Despliegue independiente por servicio	**un** proceso Django + workers: monolito modular
Bus o registro de servicios	no existe; hay eventos en memoria y una cola de exportación

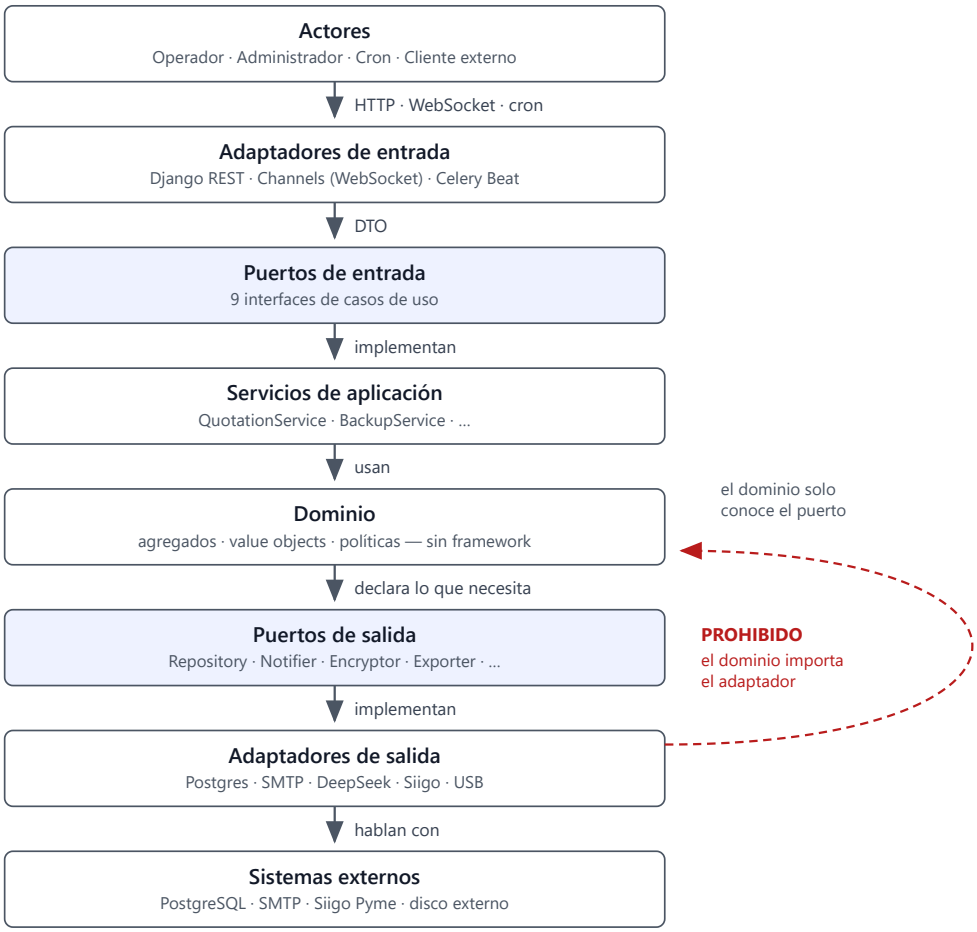
Lo único que sale por red es hacia afuera (DeepSeek por HTTPS, SMTP) o por archivo (Siigo). Eso no convierte al sistema en orientado a servicios: lo convierte en un cliente de servicios ajenos.

La confusión más frecuente: el diagrama está lleno de la palabra «service» (`QuotationService` , `BackupService` , «service» en el dominio). En DDD y en hexagonal, un **servicio de aplicación** es el objeto que orquesta un caso de uso dentro del proceso; un **servicio** en SOA es una unidad desplegable que se invoca por red. Misma palabra, conceptos distintos.

Test rápido para cerrar la discusión: si el módulo de cotizaciones se cayera, ¿el de respaldos seguiría respondiendo desde otro proceso? En IMSI, no: comparten proceso y base. Eso descarta SOA y microservicios.

---

3. La regla que hace que funcione: dirección de dependencias



Flechas que se pueden escribir y flechas que no:

Permitido	Prohibido
adaptador → puerto	dominio → adaptador
servicio de aplicación → dominio	dominio → Django / SQLAlchemy / requests
servicio de aplicación → puerto de salida	puerto de salida → adaptador
adaptador → DTO → puerto	entidad del dominio expuesta en la API

Si el dominio importa `django`, `psycopg`, `openai` o `requests`, el estilo está roto aunque el diagrama siga siendo bonito. Esa es la prueba que hace un evaluador en 30 segundos: mira los imports del paquete `domain/`.

La línea punteada del diagrama es la trampa: el dominio **declara** el puerto de salida, pero nunca importa el adaptador que lo implementa. Ese es el único contrato que hay que respetar para que el estilo sea real.

---

4. ¿Por qué hexagonal y no otra cosa?

Porque no es "el mejor modelo" en abstracto: es el que mejor encaja con las restricciones **de este** proyecto. Hexagonal no gana por ser elegante; gana porque los requisitos ya obligan a tener piezas intercambiables. Cada fila sale de una cita del `Analysis.pdf` o del `Design.pdf`.

4.1 Las razones, atadas a requisito

Restricción del proyecto	Qué exige	Cómo lo resuelve el hexagonal	Sin hexagonal
NFR-06-02: interoperabilidad con Siigo "through a flat-file export mechanism … CSV and TXT … UTF-8 without BOM"	El formato de Siigo es una traducción, no una regla de negocio	Va en un adaptador (`SiigoPymeFlatFileExporter`) detrás del puerto `AccountingExporter`	La generación del layout termina en el servicio o en la vista; un cambio de versión de Siigo toca lógica de negocio
IA reemplazable: DeepSeek hoy, Ollama como alternativa	Precio y fecha son **algoritmos**, no infraestructura	Puertos `PriceEstimator` y `DispatchDateEstimator` ; el proveedor es un adaptador	El cliente HTTP y el prompt se mezclan con el cálculo del rango; probar el rango exige red
NFR-05-02: tests "executed exclusively on synthetic data", cobertura "meet or exceed a minimum threshold of 70%"	El dominio tiene que probarse **sin PostgreSQL**	Reglas en entidades y políticas; los repositorios son puertos	Cobertura inalcanzable: cada test de regla levanta una base de datos
NFR-05-01: Strategy para fecha de despacho y rango de precios, Observer para eventos de dominio, Factory, Singleton	Los patrones son parte de la nota	Los puertos **son** Strategy; los eventos de dominio son Observer; `Quotation.create()` es Factory; el pool es Singleton	Hay que injertar los cuatro patrones a mano sobre código acoplado, que es justo lo que el requisito intenta evitar
NFR-04-02: aprobación, lotes de exportación y conciliación "within ACID-compliant database transactions … the PostgreSQL database must automatically roll back the entire transaction"	La transacción es un detalle de infraestructura	Unit of Work en el servicio de aplicación; `ATOMIC_REQUESTS` en el adaptador	La transacción se reparte entre controladores y se rompe el rollback con el primer `save()` suelto
Tres entradas reales: REST, WebSocket, cron	Los mismos casos de uso, tres transportes	Un adaptador de entrada por canal; un solo servicio	La lógica del chatbot queda pegada al consumidor de Channels
NFR-07-02: consultas estables con "millions of rows"	Read models separados del modelo de negocio	Proyecciones y vistas materializadas como adaptadores de lectura	Las consultas de reporte contaminan el dominio
NFR-08-01 y NFR-08-02: auditoría inmutable, retención legal, Habeas Data	La auditoría es un aspecto de aplicación	Interceptor de auditoría sobre los casos de uso	Se filtra a mano en cada caso de uso y se olvida en uno
Backups en USB o disco externo, con posible NAS futuro	El destino es intercambiable	Puerto `BackupStorage` , adaptador `ExternalSecureStorage`	La ruta del disco externo aparece en el servicio de respaldo

4.2 La prueba objetiva que se puede defender

No es una opinión: es un cambio concreto.

- Cambiar de proveedor de IA → **un** adaptador, **cero** tests de dominio.
- Cambiar el destino del respaldo → **un** adaptador.

- Cambiar el formato de Siigo → **un** adaptador (`SiigoPymeFlatFileExporter`).

En el diagrama, cada flecha puerto → adaptador es un punto de cambio aislado. Esa es la frase que resume la defensa: *"el dominio es lo único que el negocio no puede cambiar sin cambiar de negocio; todo lo demás es reemplazable"*.

4.3 Lo que cuesta (y nadie lo menciona)

Coste	Detalle
Más archivos por caso de uso	DTO, puerto, servicio, mapper, adaptador. Una pantalla CRUD que en Django son 30 líneas, aquí son 150
Dos conversiones por request	dict → DTO → entidad → DTO, más el coste de serializar
La regla no la impone el compilador	Un `import django` dentro de `domain/` lo rompe todo en silencio. Sin un test de arquitectura (import-linter o un script que revise imports), el estilo se degrada solo
Uniformidad forzada	Catálogo y clientes son CRUD y no necesitan dominio rico, pero hacerlos distintos crea dos convenciones; se acepta el sobrecoste para tener una sola

4.4 Cuándo **no** sería la mejor elección

- Si Siigo tuviera API estable, un proveedor de IA fijo y un solo canal de entrada: tres capas bastaban.
- Si el sistema fuera CRUD puro, sin rangos de precio, inmutabilidad, anticipos ni secuencia de O.T.
- Si el equipo cambiara de proyecto cada mes: la inversión en indirección no se amortiza.

En este caso no aplica ninguna: hay cuatro integraciones externas cambiantes, cinco módulos con reglas reales y un requisito explícito de patrones. Y hay una parte de la respuesta que conviene decir en voz alta: **el curso pide esa estructura** (NFR-05-01). Hexagonal es la forma más limpia de cumplirlo sin pegatinas, no la única.

---

5. Los siete conceptos del diagrama

Concepto	Qué es	En el diagrama
Actor	quien inicia la acción	Operator, System Admin, Cron / Task Scheduler, External Customer
Adaptador de entrada	traduce un protocolo externo a una llamada al puerto	controladores Django REST, WebSocket de Channels, disparadores de Celery Beat
Puerto de entrada	interfaz que declara qué puede pedir el exterior	`«interfaces» Quotation Use Cases` , etc.
Servicio de aplicación	orquesta un caso de uso: carga, valida, llama al dominio, persiste	`QuotationService` , `BackupService`
Dominio	reglas e invariantes; no conoce frameworks	`Quotation` , `Customer` , `WorkOrder` , `Money` , `PriceRange` , `PasswordPolicy` , `BackupPolicy`

Concepto	Qué es	En el diagrama
Puerto de salida	interfaz que declara qué necesita el sistema	`QuotationRepository`, `Notifier`, `Encryptor`, `AccountingExporter`
Adaptador de salida	implementación concreta	`PostgresQuotationRepository`, `EmailNotifier`, `AesEncryptor`, `SiigoPymeFlatFileExporter`

Y uno transversal: el **Mapper** (`Domain Entity ↔ SQL Row`). Es lo que impide que la entidad se convierta en fila de tabla y que la tabla se convierta en entidad.

---

6. El diagrama entregado, banda por banda

Tomado del `.drawio` actual (199 nodos, 93 aristas). Se lee de izquierda a derecha.

6.1 Actores (x≈0)

- `Operator (Internal User)` — secretaria/contabilidad; entra por la interfaz web.
- `System Admin (Superadmin and Admin)` — gestiona cuentas, respaldos, catálogo.
- `Cron / Task Scheduler` — no es humano: dispara el respaldo.
- `External Customer / Requester` — cliente potencial; entra por el portal público y el chatbot.

6.2 Adaptadores primarios (10)

Son la puerta real de entrada, y cada uno nombra su tecnología:

Adaptador	Tecnología declarada
Chatbot Controllers	WebSocket API / Django Channels
Auth Controllers	Django REST API / SimpleJWT
User Account Controllers	Django REST API
Quotation Controllers	Django REST API
WorkOrder Controllers	Django REST API
Customer Controllers	Django REST API
Backup Controllers	Django REST API
Accounting Export Controllers	Django REST API
Product Controllers	Django REST API
Scheduled Backup Triggers	Celery Beat Scheduler

Cada arista de actor a adaptador está etiquetada con el caso: `login`, `customer CRUD`, `user management`, `backup execution & restore`, `cron tick`, `accounting export`, `product CRUD`, `product catalog (read-only)`, `chat messages`.

6.3 Puertos de entrada (9 interfaces)

`Chat Session`, `Auth (authentication & sessions)`, `User Account (CRUD & role assignment)`, `Quotation`, `Work Order (lifecycle & status tracking)`, `Customer (CRUD, profile & commercial notes)`, `Backup (execution & restore)`, `Accounting Export (batches & flat files)`, `Product`.

Se nombran por **capacidad del negocio**, no por endpoint ni por pantalla. Eso es lo que permite que un mismo puerto lo invoquen REST, WebSocket y cron.

6.4 Servicios de aplicación (9)

`AuthService`, `UserAccountService`, `ChatSessionService`, `QuotationService`, `WorkOrderService`, `CustomerService`, `BackupService`, `AccountingExportService`, `ProductService`.

Cada uno implementa su puerto (`implements`) y usa el dominio (`uses`). Lo que **no** hace: formatear JSON, abrir conexiones, firmar tokens. Eso es del adaptador y del puerto de salida.

6.5 Dominio

Agregados: `User`, `Customer` (abstracto) con `NaturalPerson` y `LegalEntity`, `Quotation`, `WorkOrder`, `BackupFile`, `ExportBatch`, `Product`.

Entidades: `ChatSession`, `QuotationItem`, `WorkOrderItem`, `CustomerNote`, `AuditEntry`.

Value objects: `Role`, `Money`, `PriceRange`.

Reglas de dominio: `PasswordPolicy`, `BackupPolicy`.

Servicios de dominio: `WorkOrderNumberGenerator` (el consecutivo único no es responsabilidad de una entidad).

Nota: en la versión anterior del diagrama `BackupFile` y `ExportBatch` eran `«entity»`; ahora son `«aggregate root»`. Es correcto: ambos tienen ciclo de vida propio y se persisten como raíz de su agregado.

6.6 Puertos de salida (20 interfaces, 5 familias)

Familia	Interfaces
Seguridad	`PasswordHasher`, `TokenIssuer`, `TwoFactorProvider`
Notificaciones y auditoría	`Notifier`, `AuditLogger`
Servicios de negocio	`PriceEstimator`, `ConversationalAssistant`, `DispatchDateEstimator`, `AccountingExporter`, `QuotationDocumentGenerator`
Respaldo y almacenamiento	`BackupSource`, `Encryptor`, `HashCalculator`, `BackupStorage`, `FileStorage`
Persistencia	`UserRepository`, `CustomerRepository`, `QuotationRepository`, `WorkOrderRepository`, `BackupRepository`, `ProductRepository`, `ExportBatchRepository`

6.7 Adaptadores secundarios (20)

`BcryptPasswordHasher`, `JwtTokenIssuer`, `TOTPTwoFactorProvider`, `EmailNotifier (SMTP)`, `DatabaseAuditLogger`, `AiPriceEstimator (DeepSeek-V4.1)`, `DeepSeekConversationalAssistant (DeepSeek-V4.1)`, `MedianDispatchDateEstimator`, `SiigoPymeFlatFileExporter`, `PdfQuotationGenerator`, `PostgresDatabaseSource`,

`AesEncryptor`, `Sha256HashCalculator`, `ExternalSecureStorage`, `LocalFileStorage`, y los siete `Postgres*Repository`.

Nombres que valen la pena defender: `MedianDispatchDateEstimator` dice **cómo** se calcula la fecha (mediana histórica, FR-02-06); `ExternalSecureStorage` distingue el disco externo del `LocalFileStorage` interno.

6.8 Sistemas externos

`SMTP Server`, `DeepSeek-V4.1 API`, `Siigo Pyme System`, `External USB / Local Disk`, `PostgreSQL Database`.

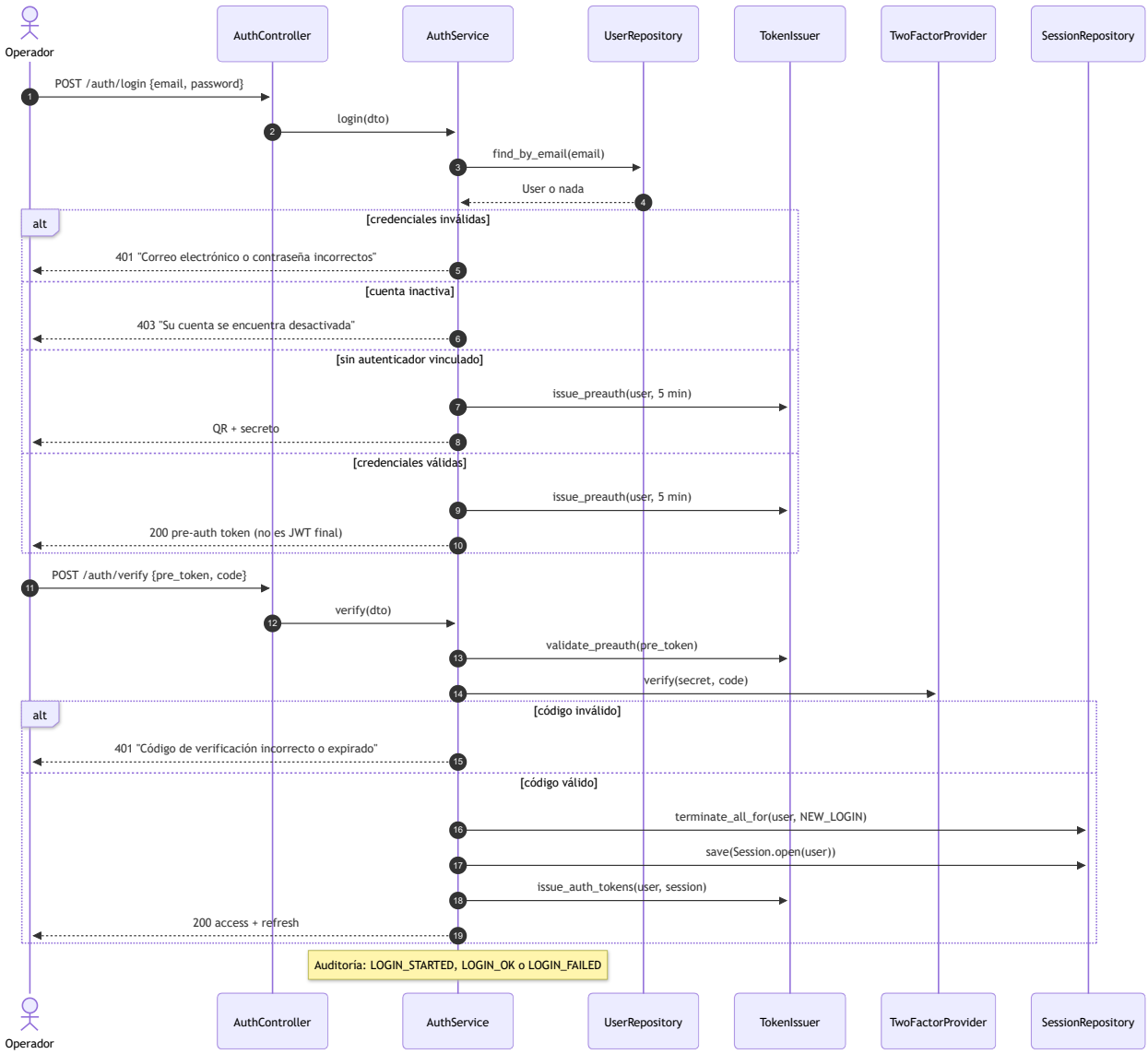
El cambio de `AWS S3` a `External USB / Local Disk` en la última versión alinea el diagrama con el diseño real: el respaldo va a un disco que alguien conecta, no a la nube.

---

7. Cómo se ve terminado: flujos con pseudocódigo

Mismo formato en los cinco: **entrada → adaptador → puerto → servicio → dominio → puertos de salida → adaptadores**. En el pseudocódigo, lo que está entre corchetes angulares `<...>` es intercambiable: esa es la gracia del hexágono.

7.1 Login con 2FA y sesión única



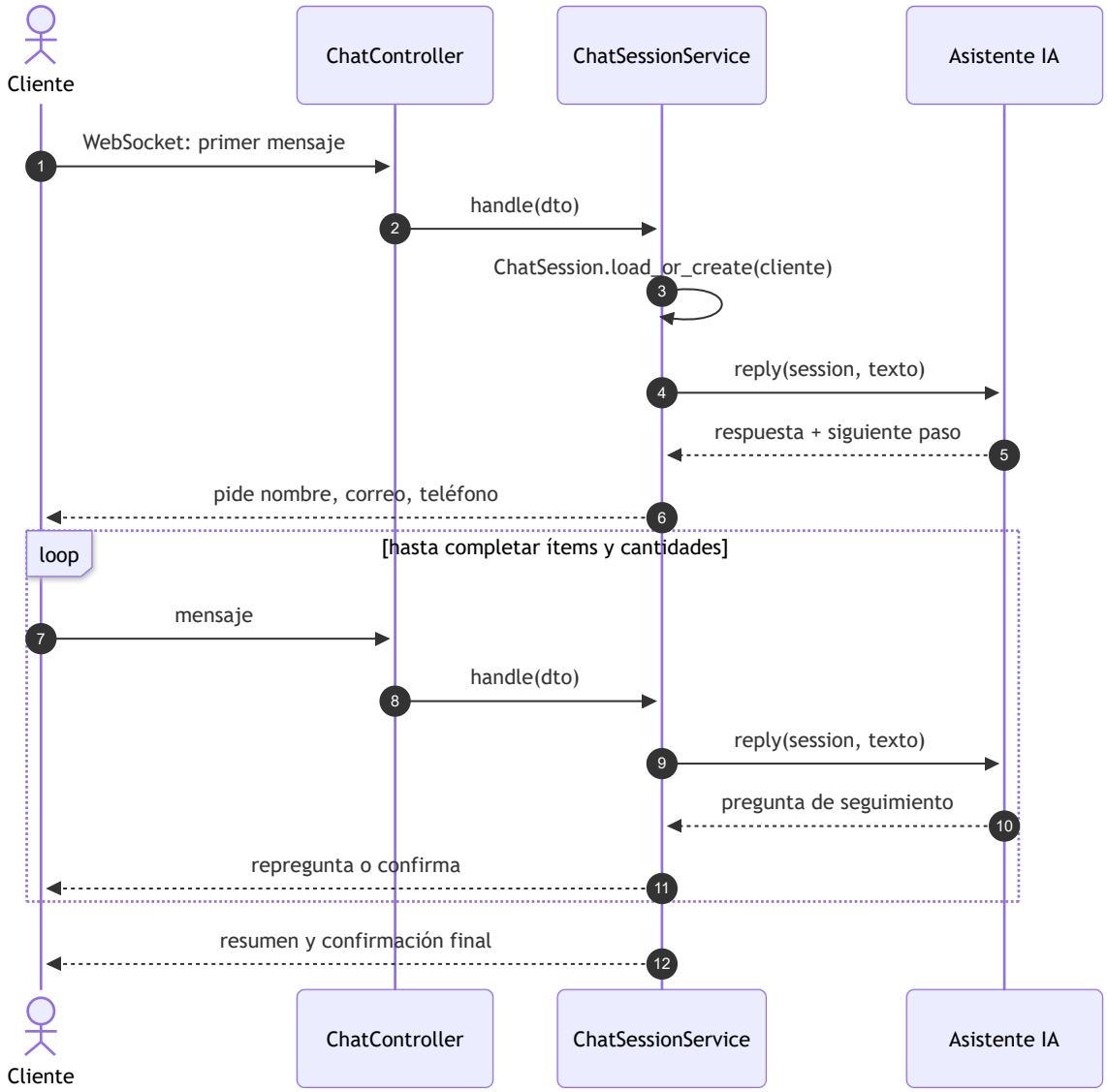
```
POST /api/v1/auth/login  {email, password}
  └─ AuthControllers.login(dto)
       auth: AuthUseCases = AuthService(...)          # puerto de entrada
       return auth.login(dto)                          # devuelve pre-auth token

AuthService.login(dto):
    user = UserRepository.find_by_email(dto.email)     # PUERTO de salida
    if user is None or not PasswordHasher.matches(dto.password, user.hash):
        raise CredencialesInvalidas("Correo electrónico o contraseña incorrectos")
    if user.status == INACTIVO:
        raise CuentaDesactivada("Su cuenta se encuentra desactivada...")
    if user.totp is None:
        return RedirectPairing(...)                    # primera vez: QR de 5 min
    pre = TokenIssuer.issue_preauth(user)              # token temporal, NO JWT final
    AuditLogger.log(LOGIN_STARTED, user)
    return pre

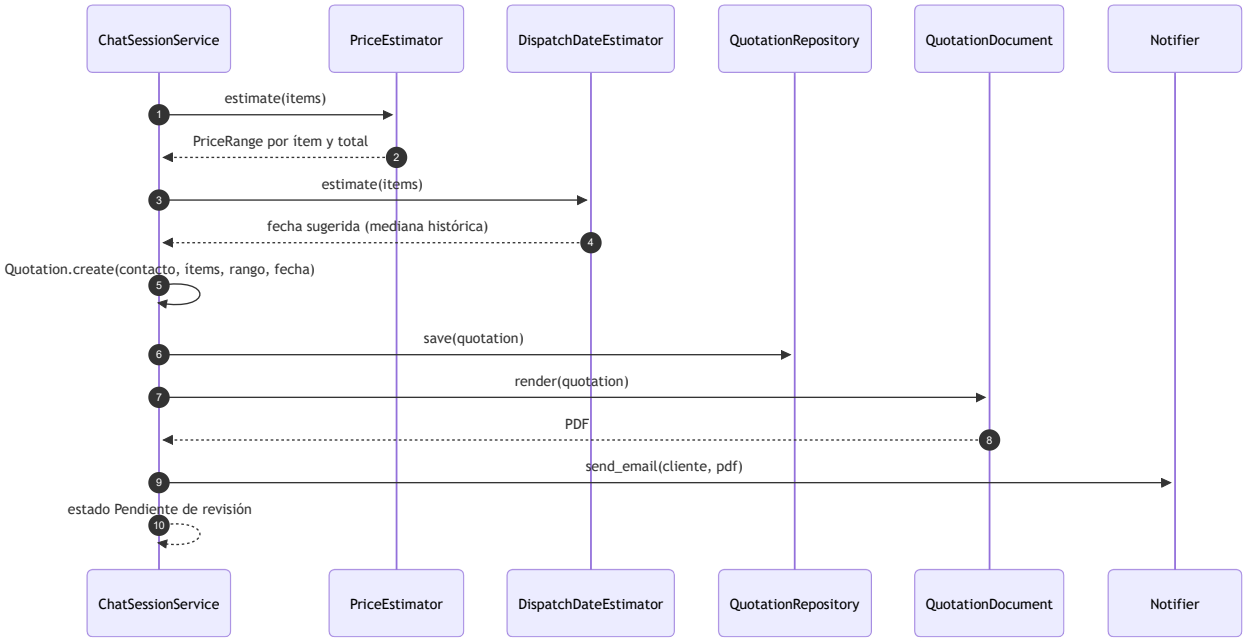
POST /api/v1/auth/verify  {pre_token, code}
  └─ AuthService.verify(dto):
       user = TokenIssuer.validate_preauth(dto.pre_token)
       if not TwoFactorProvider.verify(user.totp.secret, dto.code):
           raise CodigoInvalido("Código de verificación incorrecto o expirado")
       SessionRepository.terminate_all_for(user.id, NEW_LOGIN)   # sesión única
       session = Session.open(user)
       SessionRepository.save(session)
       AuditLogger.log(LOGIN_OK, user, session.id)
       return TokenIssuer.issue_auth_tokens(user, session.id)    # access + refresh
```

Lo que enseña: el controlador no sabe qué es un JWT ni bcrypt; el servicio no sabe qué es HTTP; el dominio (`User`, `Session`) no sabe qué es Django.

7.2 Cotización desde el chatbot (cliente externo)



Y el cierre, cuando ya están los datos y los ítems:

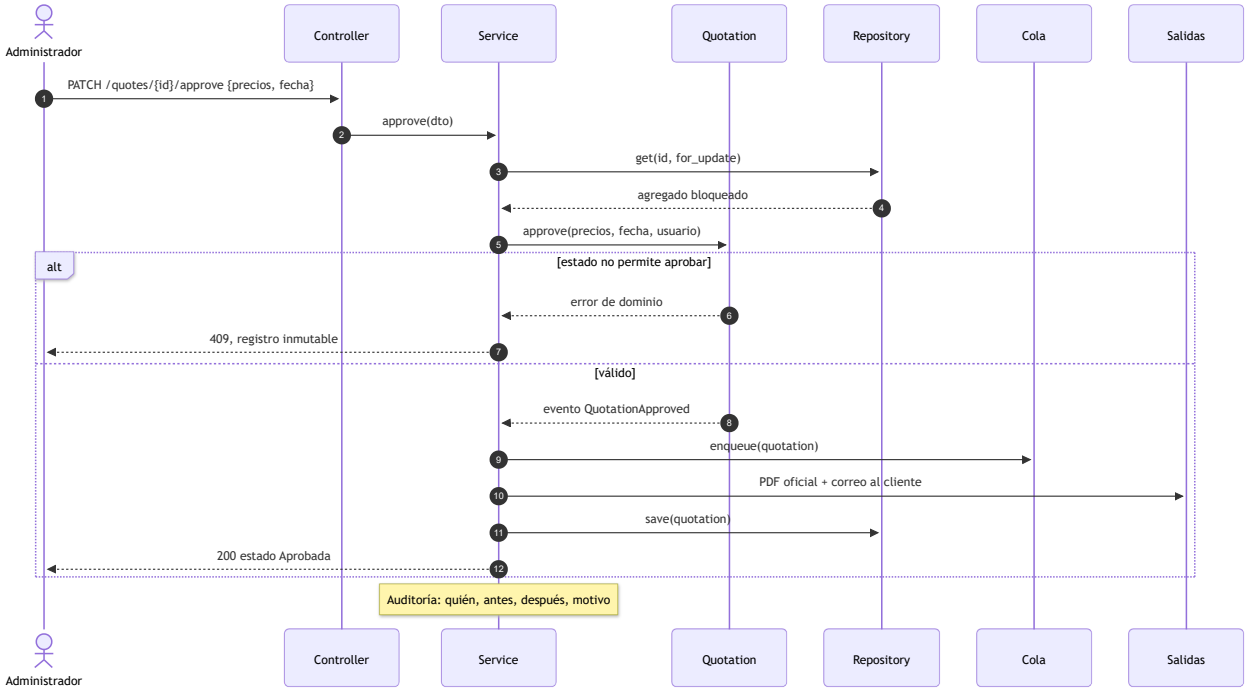


```
WS /ws/chat  {message}
  └ ChatbotControllers.on_message(dto)                # Django Channels
       chat: ChatSessionUseCases = ChatSessionService(...)
       return chat.handle(dto)

ChatSessionService.handle(dto):
    session = ChatSession.load_or_create(dto.client_id)  # estado de la conversación
    reply = ConversationalAssistant.reply(session, dto.text)  # PUERTO: DeepSeek hoy
    session.set_step(reply.next_step)                        # guiado, sin texto libre
    if not session.ready():
        return reply
    items = [QuotationItem(Product.by_code(i.code), i.qty) for i in session.items]
    ranges = PriceEstimator.estimate(items)                  # PUERTO: AiPriceEstimator
    dispatch = DispatchDateEstimator.estimate(items)         # PUERTO: mediana histórica
    quote = Quotation.create(session.contact, items, ranges, dispatch)
    QuotationRepository.save(quote)                          # PUERTO
    pdf = QuotationDocumentGenerator.render(quote)           # PUERTO
    Notifier.send_email(quote.contact.email, pdf)            # PUERTO
    AuditLogger.log(QUOTE_CREATED, actor=ANONIMO, entity=quote.id)
    return quote
```

Dos detalles que se defienden solos: la estimación de precio y la fecha son **puertos**, no llamadas directas a DeepSeek; y el estado de la conversación vive en `ChatSession`, no en la sesión HTTP.

7.3 Aprobación de cotización → cola de Siigo

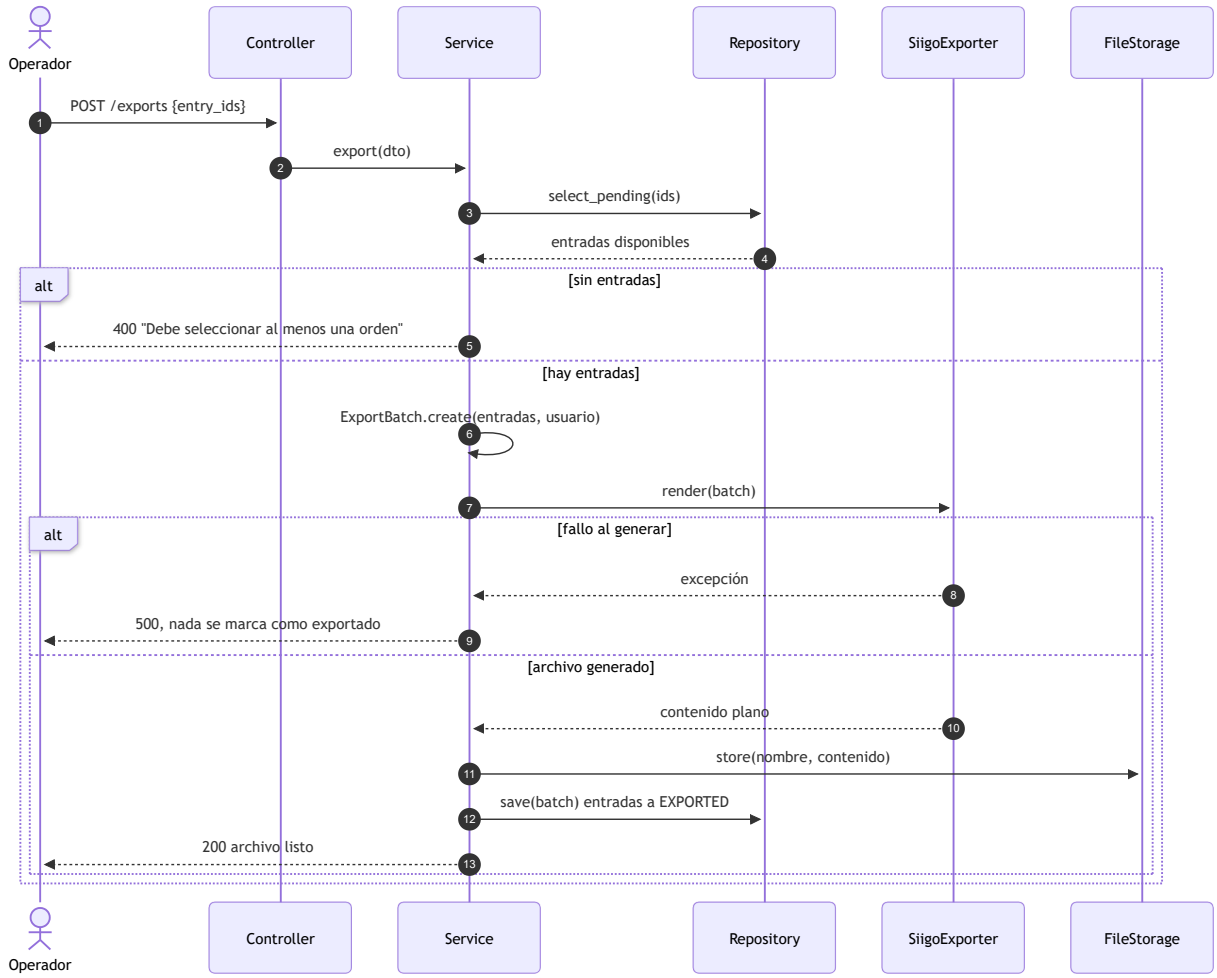



```
PATCH /api/v1/quotes/{id}/approve  {final_prices, reason}
  └ QuotationControllers.approve(dto)
       quotation: QuotationUseCases = QuotationService(...)
       return quotation.approve(dto)
```

```
QuotationService.approve(dto):
    with UnitOfWork() as uow:                        # una transacción (NFR-04-02)
        quote = QuotationRepository.get(dto.id, for_update=True)
        quote.approve(dto.final_prices, dto.dispatch_date, by=dto.user)  # dominio valida
        AuditLogger.log(APPROVED, dto.user, quote.id, before, after)
        events = quote.pull_events()                 # Observer (NFR-05-01)
        for e in events:
            if isinstance(e, QuotationApproved):
                queue.add(ProcessingQueueEntry.enqueue(quote))
                pdf = QuotationDocumentGenerator.render(quote)
                Notifier.send_email(quote.contact.email, pdf)
        uow.commit()
    return quote
```

Aquí se ven tres cosas que el diagrama insinúa pero no dibuja: **Unit of Work** (la transacción), **eventos de dominio** y el hecho de que aprobar produce tres efectos externos (cola, PDF, correo) sin que el servicio llame a tres adaptadores a mano.

7.4 Exportación a Siigo

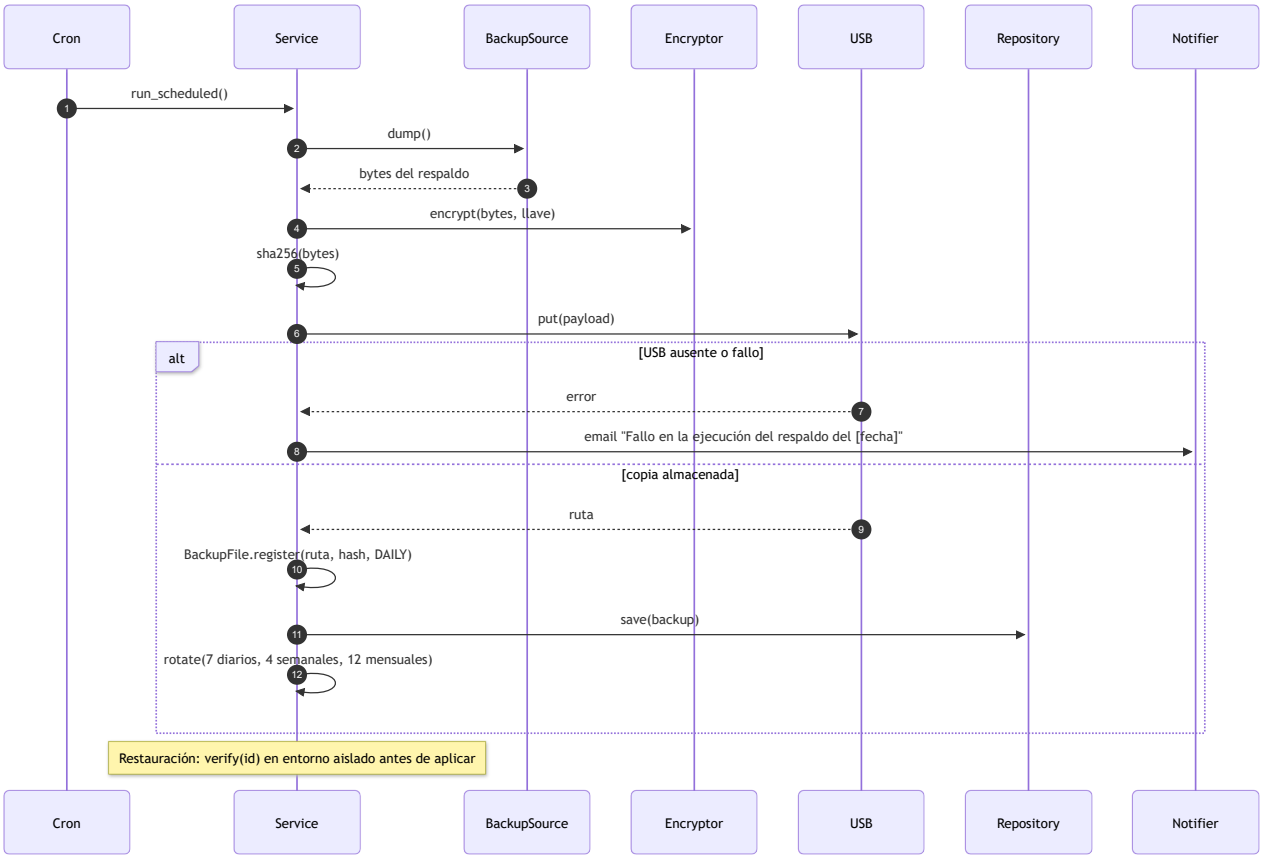


```
POST /api/v1/exports  {entry_ids}
  └─ AccountingExportControllers.export(dto):
       export: AccountingExportUseCases = AccountingExportService(...)
       return export.export(dto)

AccountingExportService.export(dto):
    entries = ExportBatchRepository.select_pending(dto.entry_ids)
    if not entries: raise NadaSeleccionado("Debe seleccionar al menos una orden")
    batch = ExportBatch.create(entries, by=dto.user)
    content = AccountingExporter.render(batch)          # PUERTO: archivo plano Siigo
    path = FileStorage.store(batch.file_name, content)  # PUERTO
    ExportBatchRepository.save(batch)                   # entradas → EXPORTED
    AuditLogger.log(EXPORTED, dto.user, batch.id, detail=len(entries))
    return path
```

La regla del análisis (FR-02-18) se cumple dentro del servicio: si falla la generación, no se marca nada como exportado. Con Unit of Work es una sola transacción; sin él, un fallo a mitad deja órdenes marcadas y sin archivo.

7.5 Respaldo programado y restauración




```
Celery Beat 18:00 → backup tarea
  └─ ScheduledBackupTriggers.run() → BackupService.run_scheduled()

BackupService.run_scheduled():
    data     = BackupSource.dump()                # PUERTO: PostgresDatabaseSource
```

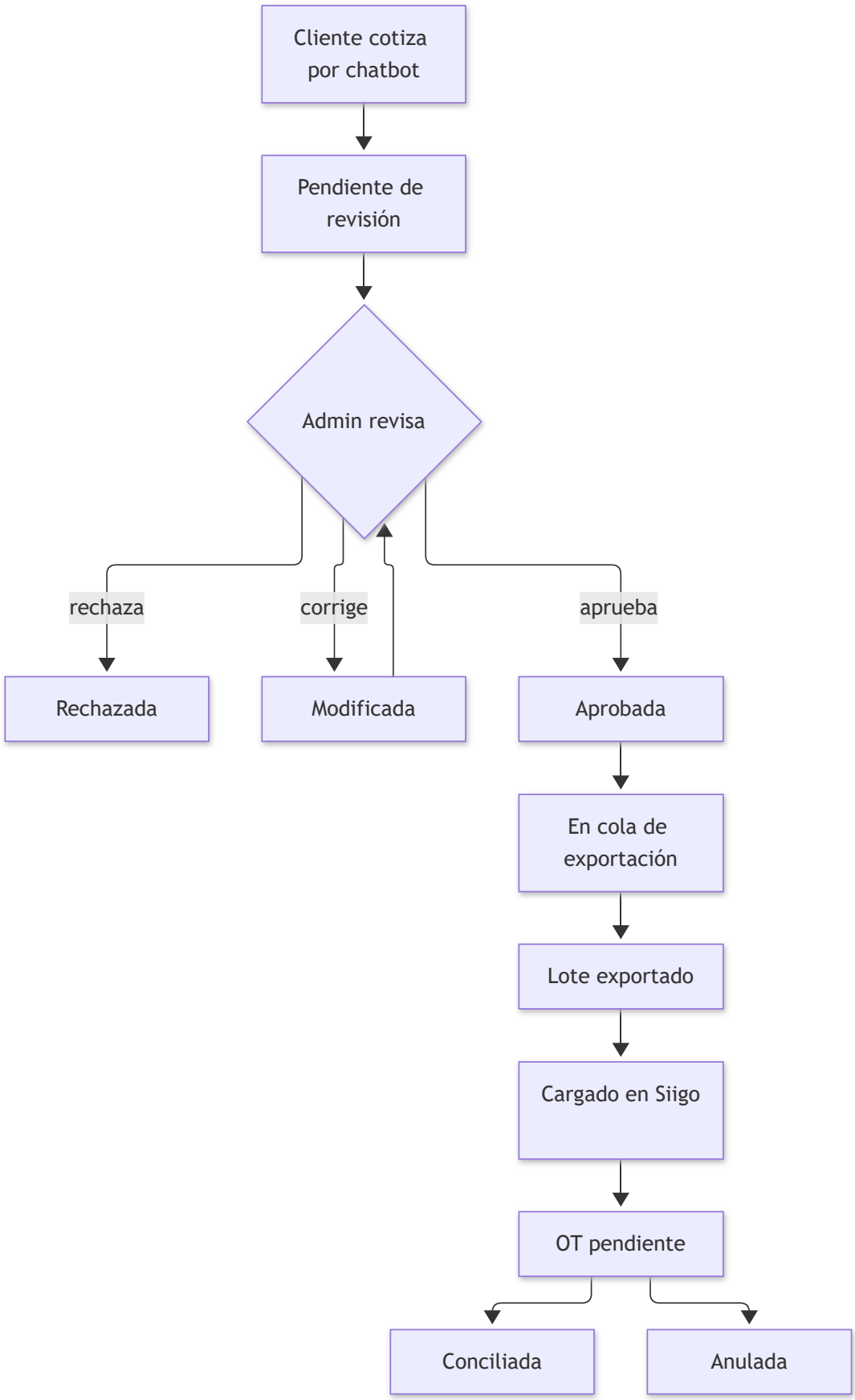
```
    payload  = Encryptor.encrypt(data, key)        # PUERTO: AesEncryptor
    digest   = HashCalculator.sha256(data)         # PUERTO: Sha256HashCalculator
    path     = BackupStorage.put(payload)          # PUERTO: ExternalSecureStorage (USB)
    backup   = BackupFile.register(path, digest, type=DAILY)
    BackupRepository.save(backup)
    BackupService.rotate()                         # 7 diarios / 4 semanales / 12 mensuales
    on failure: Notifier.send_email(admins, "Fallo en la ejecución del respaldo…")
                AuditLogger.log(BACKUP_FAILED)

POST /api/v1/backups/{id}/verify  → BackupService.verify(id)
    sandbox = BackupSource.restore_into_isolated_env(backup)   # nunca producción
    ok = sandbox.check_integrity()
    RestoreProcess.mark_verified(backup, ok)
    if not ok: raise VerificacionFallida("Error en la verificación…")
```

Este flujo es el mejor ejemplo de por qué sirve el hexágono: el dominio (`BackupFile` , `BackupPolicy`) decide *cuándo* y *qué* se conserva, mientras que *el cómo* (dump, AES, USB, SHA-256) vive en cuatro puertos distintos.

7.6 Flujo completo, de punta a punta

Estados reales del sistema, con quién actúa en cada salto:


Cliente cotiza por chatbot
Pendiente de revisión
Admin revisa
rechaza
corrige
aprueba
Rechazada
Modificada
Aprobada
En cola de exportación
Lote exportado
Cargado en Siigo
OT pendiente
Conciliada
Anulada

8. Django, REST y el ORM dentro del hexágono

Django y REST no son la arquitectura: son el **suelo donde se apoya**. Confundirlos es lo que hace que el diagrama se vea hexagonal y el código no lo sea.

8.1 Tres cosas que se confunden

Término	Qué es	Dónde vive en el diagrama
REST	Un estilo de contrato HTTP: recursos (`/quotes/{id}`), verbos (`PATCH`), códigos de estado, representaciones JSON	No es código: es la forma de la API pública
Django	El framework: ORM, migraciones, routing, settings, middleware	Infraestructura. Es el "cómo", no el "qué"
Django REST Framework	La librería que convierte Django en API: `APIView` , serializers, permisos, paginación	**Adaptador de entrada**
Hexagonal	Cómo se organiza el código propio dentro de eso	Las capas de dominio y aplicación

La frase que lo resume: **Django es reemplazable en teoría y cómodo en la práctica; el dominio no es reemplazable porque es el negocio.**

8.2 Qué exige REST (no es "devolver JSON")

- **Recursos, no acciones**: el verbo HTTP dice la intención, el recurso es el sustantivo. `PATCH /api/v1/quotes/148/approve` es aceptable en la práctica; lo importante es que el estado viva en el recurso.
- **Códigos con significado**: 400 forma inválida, 401 sin sesión, 403 sin permiso, 404 no existe, **409** estado incompatible (aprobar una cotización ya aprobada), 500 fallo del servidor.
- **Errores como datos**: `{"codigo": "QUOTE_IMMUTABLE", "mensaje": "La cotización ya fue aprobada"}` , no un HTML de Django con el stack trace.
- **Idempotencia donde toca**: repetir un `GET` o un `PUT` no cambia nada; aprobar dos veces responde 409 y no crea dos entradas en cola.
- **Nada de estado entre peticiones**: el estado va en el token (JWT) o en la base.

8.3 El serializer no es el DTO ni el validador de negocio

Su trabajo es validar **forma y tipos**:

```
class ApproveQuotationSerializer(serializers.Serializer):
    prices = serializers.DictField(child=serializers.DecimalField(max_digits=14, decimal_places=2))
    dispatch_date = serializers.DateField()
```

Lo que no va ahí:

```
def validate(self, data):
    if data["dispatch_date"] < date.today():   # regla de negocio en la frontera
        raise ValidationError("fecha inválida")
```

```
    if self.instance.state == "approved":      # invariante del agregado
        raise ValidationError("ya aprobada")
```

Esas dos reglas pertenecen a `Quotation.approve()`. Duplicarlas en el serializer crea dos verdades que un día se contradicen.

8.4 Una petición, de punta a punta

```
# 1) ADAPTADOR DE ENTRADA: solo traduce HTTP <-> DTO
class ApproveQuotationView(APIView):
    permission_classes = [IsAdminRole]
    def patch(self, request, quote_id):
        dto = ApproveQuotationSerializer(data=request.data)
        dto.is_valid(raise_exception=True)          # forma, no negocio
        result = self.approval_use_case.approve(     # puerto de entrada
            quote_id, ApproveQuotationDTO(**dto.validated_data), request.user
        )
        return Response(QuotationResponse.from_entity(result), status=200)
```

```
# 2) SERVICIO DE APLICACIÓN: orquesta, no decide reglas
class ApprovalService(ApproveQuotationUseCase):
    def approve(self, quote_id, dto, user):
        with self.uow:                              # transacción (NFR-04-02)
            q = self.quotes.get_for_update(quote_id)
            q.approve(dto.prices, dto.dispatch_date, user)   # el dominio decide
            self.quotes.save(q)
            self.events.publish(q.pull_events())    # Observer (NFR-05-01)
        return q
```

```
# 3) ADAPTADOR DE SALIDA: ORM <-> entidad
class DjangoQuotationRepository(QuotationRepository):
    def get_for_update(self, quote_id):
        row = QuotationRow.objects.select_for_update().get(pk=quote_id)
        return to_entity(row)                       # mapper explícito
```

En la vista no aparece `select_for_update`, ni `transaction.atomic`, ni `QuotationRow`. Todo eso es infraestructura y vive en las otras dos capas: la vista son cinco líneas porque no sabe nada del negocio.

8.5 Qué regala Django (y no hay que reescribir)

Te da	Dónde se usa
ORM y migraciones	Implementación de los repositorios
Routing y parsing	`urls.py`, `APIView`
Autenticación y permisos	`SimpleJWT` como adaptador de `TokenIssuer`; `IsAdminRole`

Te da	Dónde se usa
Transacciones	`ATOMIC_REQUESTS` y `select_for_update()` en el Unit of Work
Hashers	bcrypt coste 12 (NFR-01-02) vía `BcryptPasswordHasher`
Testing	`APIClient` para integración; el dominio se prueba sin él

8.6 Qué es el ORM y qué hace por ti

El ORM (Object-Relational Mapping) es la capa que traduce entre tus objetos de Python y las tablas de SQL. Tú escribes `QuotationRow.objects.filter(state="pending")` y él genera la consulta, ejecuta, hidrata los objetos y gestiona las relaciones. No es la base de datos: es un **intérprete** entre dos mundos.

Lo que hace bien:

- **CRUD y consultas**: filtros, orden, agregaciones, joins.
- **Relaciones**: `ForeignKey`, `ManyToManyField`, y el acceso `cliente.cotizaciones.all()`.
- **Migraciones**: el esquema versionado junto al código.
- **Transacciones**: `transaction.atomic()`, y locking con `select_for_update()`.
- **Identidad de objetos**: dentro de una sesión, la misma fila devuelve el mismo objeto.

Lo que **no** hace, aunque lo parezca:

- **No protege invariantes de negocio.** El ORM no sabe que una cotización aprobada es inmutable ni que una O.T. no salta de `Pendiente` a `Conciliada`.
- **No conoce tu agregado.** Un `QuotationRow` es una fila; una `Quotation` es un agregado con reglas. Son dos cosas distintas con el mismo nombre.
- **No decide transacciones de negocio.** Sabe abrir y cerrar una transacción; no sabe dónde empieza y termina un caso de uso.

8.7 Dónde el ORM hace daño

Síntoma	Causa	Arreglo
1 consulta + N por cada fila	Lazy loading dentro de un bucle (N+1)	`select_related()` para FK, `prefetch_related()` para M2M
Un queryset “mágicamente” ejecutado	Evaluación perezosa: el queryset no consulta hasta que se recorre	Materializarlo donde se decide (`list(...)`) y no en la plantilla
Reglas duplicadas	Lógica en métodos del modelo Django (modelo activo)	La regla va al agregado; el modelo solo se hidrata
Consulta lenta con millones de filas (NFR-07-02)	Filtros sobre columnas sin índice, o `.all()` en reportes	Índices, `.values()` para proyecciones, vistas materializadas
Lock olvidado en concurrencia	Dos aprobaciones simultáneas	`select_for_update()` dentro del Unit of Work
Migración que borra datos	`makemigrations` sin revisar el diff	Revisar el SQL, migración de datos antes del `AlterField`

Regla de oro: **un `QuerySet` nunca cruza la frontera del repositorio.** El repositorio devuelve agregados o proyecciones, no querysets perezosos que el servicio pueda seguir encadenando. Si el servicio puede escribir

`self.quotes.filter(...)`, el repositorio no está haciendo su trabajo.

8.8 ¿Por qué repositorios y no `Model.objects` en el servicio?

Un repositorio es la **representación de la colección de agregados**: para el servicio, `self.quotes` es una colección de cotizaciones con métodos de negocio, no una tabla. El ORM es un detalle de esa implementación.

```python
# puerto de salida: habla el idioma del negocio
class QuotationRepository(Protocol):
    def get_for_update(self, quote_id: QuotationId) -> Quotation: ...
    def find_pending_review(self) -> list[Quotation]: ...
    def next_sequence(self) -> int: ...
```

```python
# adaptador real: ORM dentro, entidad fuera
class DjangoQuotationRepository:
    def find_pending_review(self):
        rows = QuotationRow.objects.filter(state="pending").select_related("customer")
        return [to_entity(r) for r in rows]
```

```python
# adaptador de prueba: cero base de datos
class InMemoryQuotationRepository:
    def __init__(self, quotes): self._by_id = {q.id: q for q in quotes}
    def get_for_update(self, quote_id): return self._by_id[quote_id]
```

Seis razones concretas:

Razón	Qué gana el proyecto
Aísla el ORM	Ningún servicio importa `django.db`. Cambiar de ORM o de motor toca solo el adaptador
Testabilidad	Un `InMemoryRepository` permite probar reglas sin PostgreSQL: requisito de NFR-05-02 (cobertura 70 % con datos sintéticos)
Lenguaje del dominio	`find_pending_review()` dice qué necesita el negocio; `objects.filter(state="pending")` dice cómo se consulta
Rendimiento localizado	`select_related`, `only()`, `.values()` y los índices viven en un solo archivo por agregado (NFR-07-02: millones de filas)
Concurrencia	`select_for_update()` está donde debe: el repositorio, dentro del Unit of Work
Auditoría de consultas	Si todo acceso pasa por métodos nombrados, se pueden contar y optimizar; con `Model.objects` disperso, no

Repositorio no es DAO. El DAO es por tabla; el repositorio es por **agregado**. Por eso el diagrama tiene siete repositorios Postgres y no uno por tabla: cada uno devuelve un agregado completo (`Quotation`, `WorkOrder`, `BackupFile`, `ExportBatch`, `User`, `Session`, `AuditEntry`).

Los cuatro anti-patrones que lo convierten en un estorbo:

1. **Devolver `QuerySet`** en vez de agregados: el servicio vuelve a encadenar consultas y el aislamiento se perdió.
2. **Un método por combinación de filtros de pantalla**: acabas construyendo un query builder con otro nombre. Para eso están los read models.
3. **Reglas de negocio dentro del repositorio**: el repositorio guarda y recupera; no decide si algo es aprobable.
4. **Un repositorio por tabla en vez de por agregado**: dos repositorios que guardan partes del mismo agregado permiten estados a medias.

Y el coste, para ser honestos: un archivo extra por agregado, un mapper y, en CRUD simple, ceremonia que no se nota. Se paga porque el resto del sistema gana prueba sin base y una sola puerta de entrada al ORM.

8.9 Cómo se vería en carpetas (Django)

```
mi_modulo/
├── domain/                    # sin imports de Django
│   ├── quotation.py           # agregado + invariantes
│   ├── value_objects.py       # Money, PriceRange, Role
│   └── policies.py            # PasswordPolicy, BackupPolicy
├── application/
│   ├── ports/inbound/         # QuotationUseCases (interfaz)
│   ├── ports/outbound/        # QuotationRepository, Notifier (interfaces)
│   └── services/              # QuotationService (implementa el puerto de entrada)
├── adapters/
│   ├── inbound/               # views.py, serializers (DTO), consumers.py, tasks.py
│   └── outbound/              # postgres_quotation_repository.py, email_notifier.py
└── mappers/                   # entity ⇔ row, entity ⇔ DTO
```

8.10 Reglas de oro al escribir código

- `domain/` no importa nada de `django`. Si necesitas `timezone.now()`, pásalo como argumento o defínelo en un `Clock` (puerto).
- Las vistas no contienen lógica: convierten DTO y llaman a un puerto de entrada.
- Los serializers de DRF son **DTO**, no entidades; nunca devuelvas un agregado.
- Los repositorios devuelven agregados completos, no `QuerySet`.
- Un solo `UnitOfWork` por caso de uso; los servicios no abren transacciones sueltas.

---

9. Las cinco trampas que rompen el hexágono

Trampa	Cómo se detecta	Efecto
Usar `models.Model` como entidad de dominio	`domain/` importa `django.db`	el dominio depende del ORM; las pruebas necesitan base de datos
Devolver el agregado en la API	el JSON tiene campos internos o `id` de tabla	acopla el contrato externo al modelo interno

Trampa	Cómo se detecta	Efecto
Poner reglas en el serializer	`validate_*` con reglas de negocio	la regla se duplica en cada entrada (REST, WS, cron)
Servicio que llama a `requests` o `boto3`	imports de red en `application/`	no se puede probar sin red; no es sustituible
Sin Unit of Work	`save()` dispersos por el servicio	un fallo deja datos a medias (NFR-04-02 exige rollback)

---

10. Lo que al diagrama le falta (crítica constructiva)

Útil para defenderlo y para saber qué preguntarían:

1. **Factory** (exigido por NFR-05-01) no aparece. Los agregados se crean con `Quotation.create(...)` / `WorkOrder.create(...)`, que es un *factory method*, pero el diagrama no lo rotula. Añadir un `«interface» DomainFactory` o anotar los `create()` resolvería la trazabilidad del requisito.
2. **Observer** está implícito: aprobar una cotización dispara cola, PDF y correo, pero no hay nodo de eventos (`QuotationApproved`, `BackupFailed`). Sin él, el servicio de aplicación conoce a sus tres consumidores, que es justo lo que el patrón evita.
3. **Unit of Work / transacción** no existe como nodo. NFR-04-02 lo exige explícitamente, y sin él el flujo de exportación puede quedar a medias.
4. **Autorización** no está dibujada. El RBAC (FR-01-04) se aplica como decorador en los adaptadores de entrada, o como puerto (`Authorizer`) si quieres que el servicio también valide. Decidirlo y dibujarlo.
5. **Sesión y 2FA**: `AuthService` usa `Session`, pero no hay `SessionRepository` en los puertos de salida; aparece dentro de Persistencia solo implícitamente.
6. **Singleton** (pool de conexiones) no se dibuja: es de infraestructura, pero si el NFR lo nombra, conviene una nota en el adaptador de persistencia.
7. **Configuración e inyección**: no se ve quién construye `QuotationService` con qué adaptadores. Un bloque "composition root" aclara cómo se sustituye DeepSeek por otro proveedor sin tocar código.

Ninguna de estas es un error del estilo; son huecos de representación. La más importante para la nota es la 2 y la 3, porque tocan requisitos explícitos (NFR-05-01 y NFR-04-02).

---

11. Checklist para defenderlo en 30 segundos

1. Nombre del estilo: **arquitectura hexagonal (puertos y adaptadores)**. Si preguntan por despliegue, un solo despliegue con módulos internos.
2. Regla única: las dependencias apuntan al dominio; el dominio no importa nada.
3. Puertos = interfaces; adaptadores = implementaciones; DTO y mapper en la frontera.
4. Tres entradas reales: REST (Django + SimpleJWT), WebSocket (Channels), cron (Celery Beat).
5. Cinco familias de salida: seguridad, notificaciones/auditoría, servicios de negocio, respaldo, persistencia.
6. Lo cambiante está fuera del dominio: DeepSeek, Siigo, USB, PostgreSQL.

7. Lo que se prueba sin infraestructura es lo que importa: invariantes de `Quotation`, `WorkOrder`, `BackupPolicy`.
8. Los cuatro patrones del NFR se ubican: Strategy (`PriceEstimator`, `DispatchDateEstimator`), Observer (eventos al aprobar), Factory (`create` de agregados), Singleton (pool).
9. Lo que falta y se reconoce: eventos dibujados, Unit of Work, autorización.

---

12. Relación con el macro (documento aparte, no es entregable)

En el workspace hay un diagrama propio, `Arquitectura_Modular_Hexagonal.drawio` (página "00 Macro - Monolito modular"), que describe el despliegue completo: nginx con TLS, frontend Next.js, un proceso Django + DRF con Gunicorn y Celery, y quince módulos con fronteras estrictas; cada módulo se abre como hexágono en su página.

Dos cosas importantes:

- **Ese macro no está en la entrega.** La entrega (el repo del equipo) tiene un solo diagrama, titulado "IMSI - Hexagonal Architecture", y el Design.pdf dice literalmente "we chose to implement a Hexagonal Architecture". Ahí no aparece la palabra "monolítico" ni "modular".
- El macro sirve para responder "¿y cómo se despliega esto?" y para entender los módulos. No lo presentes como si fuera parte del entregable, y no uses la etiqueta "monolito modular" para contestar una pregunta sobre el estilo.

13. Explorador interactivo

El documento y el diagrama se pueden recorrer juntos en `herramientas/imsi-explorer.html`: un solo archivo, sin servidor, que se abre con doble clic y funciona sin internet (los mockups van incrustados).

Pestaña	Para qué sirve
Mapa	Las ocho capas del diagrama entregado, con las 79 conexiones reales y un inspector por nodo: qué es, en qué capa vive, qué consume y qué le llega
Flujos	Los seis recorridos del capítulo 7 en reproducción paso a paso: nodo activo, rastro de lo recorrido y la flecha resaltada cuando existe en el diagrama
Pantallas	Las diez pantallas por rol, con sus mockups

Los datos no están escritos a mano: `herramientas/build-explorer.py` los extrae del propio `.drawio` (102 nodos, 79 aristas) y deduce la capa de cada nodo por el panel que lo contiene. Si el equipo cambia el diagrama, se reemplaza el archivo y se vuelve a ejecutar el script; el mapa se actualiza solo.

Para defender el proyecto: abrir el mapa, elegir un flujo y reproducirlo. Cada salto que brilla corresponde a una flecha que existe en el diagrama; cuando la narración avanza y no hay flecha, es un hueco real del diseño, no un fallo del explorador. Al final de cada flujo aparece la nota con ese hueco. —

14. Cada historia de usuario, ubicada en el diagrama

La entrega tiene **61 historias** (US 1.1 a US 2.40). El catálogo del documento de casos de uso tiene 26: **20 caen en el alcance entregado** y los otros seis (auditoría financiera) vienen del borrador del Drive, así que aquí se listan los 20 con recorrido real en el diagrama. Esta tabla es la traducción directa: para cada caso, las historias que lo usan y el recorrido de nodos.

Cómo leerla: el recorrido empieza en el actor y termina en el sistema externo, pasando por adaptador de entrada → puerto → servicio → dominio → puerto de salida → adaptador de salida. Si quieres verlo animado, el explorador tiene los seis flujos principales (`imsi-explorer.html`); esta tabla cubre los 26.

CU-001 · Iniciar sesión con doble factor
Historias: US 1.1, US 1.2, US 1.3
`Operator → Auth Controllers → Auth Use Cases → AuthService → PasswordHasher / TokenIssuer / TwoFactorProvider → PostgresUserRepository → PostgreSQL`

CU-002 · Controlar sesión única y expiración
Historias: US 1.5, US 1.6
`Operator → Auth Controllers → Auth Use Cases → AuthService → (falta SessionRepository) → AuditLogger → PostgreSQL`

CU-003 · Administrar cuentas y roles
Historias: US 1.4, US 1.15, US 1.16, US 1.17, US 1.18, US 1.19, US 1.20, US 1.21
`Admin → User Account Controllers → User Account Use Cases → UserAccountService → User / Role → UserRepository → PostgresUserRepository → PostgreSQL`

CU-004 · Ejecutar el respaldo programado
Historias: US 1.7, US 1.8, US 1.9
`Cron → Scheduled Backup Triggers → Backup Use Cases → BackupService → BackupSource → Encryptor → HashCalculator → BackupStorage → External USB / Local Disk`

CU-005 · Consultar inventario y descargar copia
Historias: US 1.10, US 1.11, US 1.12
`Admin → Backup Controllers → Backup Use Cases → BackupService → BackupRepository → PostgresBackupRepository → PostgreSQL; FileStorage → ExternalSecureStorage`

CU-006 · Verificar y restaurar un respaldo
Historias: US 1.13, US 1.14
`Superadmin → Backup Controllers → Backup Use Cases → BackupService → HashCalculator + BackupStorage → prueba en entorno aislado → USB`

CU-007 · Consultar bitácora de auditoría
Historias: sin historia directa (NFR-08-01)
`Admin → (falta Audit Controllers) → AuditLogger → DatabaseAuditLogger → PostgreSQL`

CU-008 · Consultar catálogo de productos
Historias: US 2.1, US 2.2
`Operador → Product Controllers → Product Use Cases → ProductService → Product → ProductRepository → PostgresProductRepository → PostgreSQL`

CU-009 · Administrar producto y precio
Historias: US 2.3, US 2.4, US 2.5, US 2.6, US 2.7, US 2.8
`Admin → Product Controllers → Product Use Cases → ProductService → Product (precio + historial) → ProductRepository → PostgreSQL`

CU-010 · Registrar cotización por el asistente
Historias: US 2.9, US 2.10, US 2.11, US 2.12

`Cliente → Chatbot Controllers → Chat Session Use Cases → ChatSessionService → ConversationalAssistant / PriceEstimator / DispatchDateEstimator → DeepSeek → Quotation → QuotationRepository → PostgreSQL; QuotationDocumentGenerator + Notifier`

CU-011 · Consultar cotizaciones preliminares

Historias: US 2.25, US 2.26

`Admin → Quotation Controllers → Quotation Use Cases → QuotationService → QuotationRepository → PostgresQuotationRepository → PostgreSQL`

CU-012 · Revisar, ajustar y aprobar cotización

Historias: US 2.13, US 2.27, US 2.28

`Admin → Quotation Controllers → Quotation Use Cases → QuotationService → Quotation.approve → repositorio → evento → cola + PDF + correo; AuditLogger`

CU-013 · Rechazar cotización con motivo

Historias: US 2.29

`Admin → Quotation Controllers → Quotation Use Cases → QuotationService → Quotation.reject → QuotationRepository → Notifier`

CU-014 · Consultar cola de órdenes aprobadas

Historias: US 2.30, US 2.31, US 2.32

`Operador → Accounting Export Controllers → Accounting Export Use Cases → AccountingExportService → WorkOrder / ExportBatch → repositorios → PostgreSQL`

CU-015 · Generar lote para Siigo

Historias: US 2.33, US 2.34

`Operador → Accounting Export Controllers → Export Use Cases → AccountingExportService → ExportBatch → ExportBatchRepository → AccountingExporter → SiigoPymeFlatFileExporter → FileStorage → Siigo Pyme (carga manual)`

CU-016 · Registrar y consultar O.T.

Historias: US 2.35, US 2.36, US 2.37

`Operador → WorkOrder Controllers → Work Order Use Cases → WorkOrderService → WorkOrder → WorkOrderRepository → PostgresWorkOrderRepository → PostgreSQL`

CU-017 · Modificar, anular y cambiar estado

Historias: US 2.38, US 2.39, US 2.40

`Operador → WorkOrder Controllers → WorkOrderService → WorkOrder (máquina de estados) → WorkOrderNumberGenerator → WorkOrderRepository → PostgreSQL; AuditLogger`

CU-018 · Gestionar clientes

Historias: US 2.16, US 2.17, US 2.18

`Operador → Customer Controllers → Customer Use Cases → CustomerService → Customer / LegalEntity / NaturalPerson → CustomerRepository → PostgresCustomerRepository → PostgreSQL`

CU-019 · Consultar ficha 360 e historial

Historias: US 2.14, US 2.15, US 2.19, US 2.20

`Operador → Customer Controllers → Customer Use Cases → CustomerService → proyecciones de lectura (ficha, historial) → CustomerRepository → PostgreSQL`

CU-020 · Gestionar notas comerciales

Historias: US 2.21, US 2.22, US 2.23, US 2.24

`Operador → Customer Controllers → Customer Use Cases → CustomerService → CustomerNote → CustomerRepository → PostgresCustomerRepository`

14.1 Las tres historias que conviene explicar en voz alta

Si hay que elegir tres para la sustentación, son estas porque cada una muestra una propiedad distinta del estilo:

1. **US 2.27 (aprobar cotización).** Muestra el dominio decidiendo (*inmutable*) y el servicio orquestando (transacción, evento, auditoría). Si la regla estuviera en el controlador, la aprobación no podría probarse sin HTTP.
2. **US 2.34 (exportar a Siigo).** Muestra el puerto de salida: el servicio no sabe que Siigo no tiene API; solo pide «genera el archivo». Cambiar el formato toca un adaptador.
3. **US 1.1 (login con 2FA).** Muestra la cadena completa de puertos de seguridad y, de paso, un hueco real del diagrama: la sesión única necesita un `SessionRepository` que no está dibujado.

14.2 El hueco que la tabla deja ver

Cinco historias dependen de una pieza que el diagrama no tiene:

Historia	Pieza que falta
US 1.5, US 1.6	`SessionRepository` y el agregado `Session`
US 1.13, US 1.14	El entorno aislado de verificación (no hay adaptador dibujado)
US 2.30	La cola de procesamiento como agregado (hoy es un estado de la O.T.)

Eso no invalida el diseño: es lo que hay que añadir al diagrama antes de sustentar, y demuestra que la tabla de trazabilidad sirve para encontrar huecos, no solo para decorar la entrega.

14.3 Índice de las 61 historias

Cada historia, con el caso de uso que la cubre. El recorrido completo, con los nombres exactos de los nodos, está en el apartado de cada caso (arriba).

Historia	Qué pide (texto original, recortado)	Caso
US 1.1	Log in with my email address and password	CU-001
US 1.2	Link an external authenticator application using a QR code	CU-001
US 1.3	Enter the 6-digit temporary code from my authenticator app after my…	CU-001
US 1.4	Access to modules, views, and actions to depend exclusively on each…	CU-003
US 1.5	My account to maintain only one active session at a time	CU-002
US 1.6	My session to automatically close after a period of inactivity	CU-002
US 1.7	The system to automatically generate an encrypted differential backup…	CU-004

Historia	Qué pide (texto original, recortado)	Caso
US 1.8	The system to automatically rotate and purge backup versions…	CU-004
US 1.9	Receive an email notification when automated backup fails	CU-004
US 1.10	Consult the inventory of available backup copies	CU-005
US 1.11	Filter the backup inventory by various attributes	CU-005
US 1.12	Directly download a stored backup copy	CU-005
US 1.13	The system to automatically test a backup in an isolated environment…	CU-006
US 1.14	Explicitly confirm restoration of a verified backup over the main…	CU-006
US 1.15	Consult a paginated list of user accounts	CU-003
US 1.16	Filter and search user accounts	CU-003
US 1.17	Register new user accounts	CU-003
US 1.18	Modify user account details	CU-003
US 1.19	Update a user account password	CU-003
US 1.20	Deactivate user accounts without deleting them	CU-003
US 1.21	Reactivate previously deactivated user accounts	CU-003
US 2.1	Consult the product catalog	CU-008
US 2.2	Filter products in the catalog	CU-008
US 2.3	Register new products in the central catalog	CU-009
US 2.4	Modify the description and unit of measure of a product	CU-009
US 2.5	Deactivate products without deleting them	CU-009
US 2.6	Reactivate previously deactivated products	CU-009
US 2.7	Provide my contact details and required products and quantities to…	CU-009
US 2.8	Download my quote as a PDF document directly from the virtual…	CU-009
US 2.9	Receive my quote as a PDF document by email	CU-010

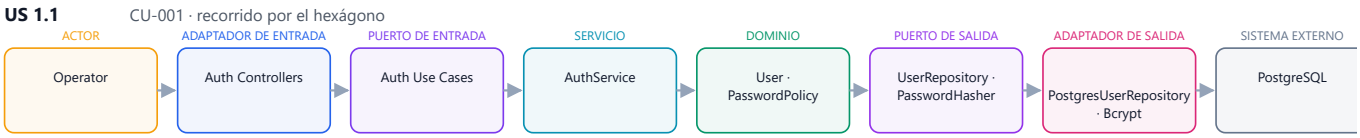
Historia	Qué pide (texto original, recortado)	Caso
US 2.10	Each quote generated by the virtual assistant to be automatically…	CU-010
US 2.11	The system to calculate and display a suggested dispatch date on…	CU-010
US 2.12	Confirm the suggested dispatch date on a preliminary quote	CU-010
US 2.13	Adjust the suggested dispatch date on a preliminary quote, or enter…	CU-012
US 2.14	Consult a paginated list of customers	CU-019
US 2.15	Filter and search customers	CU-019
US 2.16	Register new customers	CU-018
US 2.17	Modify customer information	CU-018
US 2.18	Deactivate customers without deleting them	CU-018
US 2.19	Consult a unified sheet of a customer, identified by NIT or document…	CU-019
US 2.20	Consult the chronological history of quotes and work orders for a…	CU-019
US 2.21	Consult internal commercial notes on a customer’s profile sheet	CU-020
US 2.22	Register an internal commercial note on a customer’s profile sheet	CU-020
US 2.23	Modify an internal commercial note that I recorded	CU-020
US 2.24	Delete an internal commercial note from a customer’s profile sheet	CU-020
US 2.25	Consult a paginated list of preliminary quotes generated by the…	CU-011
US 2.26	Filter preliminary quotes	CU-011
US 2.27	Approve a preliminary quote	CU-012
US 2.28	Modify a preliminary quote	CU-012
US 2.29	Reject a preliminary quote by stating a reason	CU-013
US 2.30	Approved commercial orders to automatically enter a processing queue	CU-014
US 2.31	Consult the queue of approved orders	CU-014
US 2.32	Filter orders in the queue	CU-014
US 2.33	Select multiple orders from the processing queue	CU-015

Historia	Qué pide (texto original, recortado)	Caso
US 2.34	Export selected orders into a single flat file for Siigo Pyme	CU-015
US 2.35	Consult a paginated list of work orders	CU-016
US 2.36	Filter work orders	CU-016
US 2.37	Manually register a new work order	CU-016
US 2.38	Modify an existing work order	CU-017
US 2.39	Annul a work order stating a reason	CU-017
US 2.40	Update work order status following its established workflow	CU-017

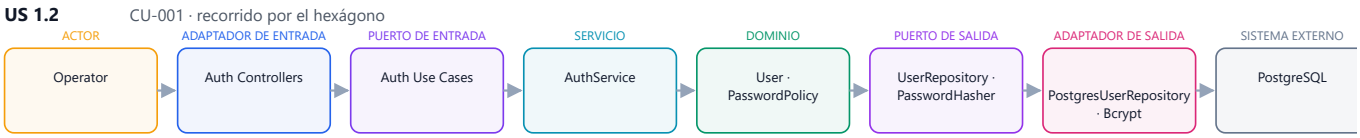
14.4 El flujo de cada historia en el hexágono

Una tira por historia: ocho estaciones, de izquierda a derecha. Las historias que comparten caso comparten recorrido; aun así está dibujado cada una.

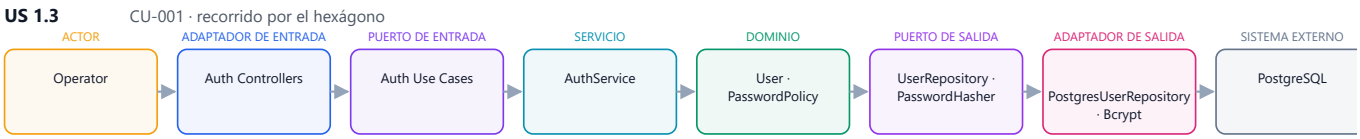
US 1.1 · Log in with my email address and…
CU-001 · Iniciar sesión 2FA



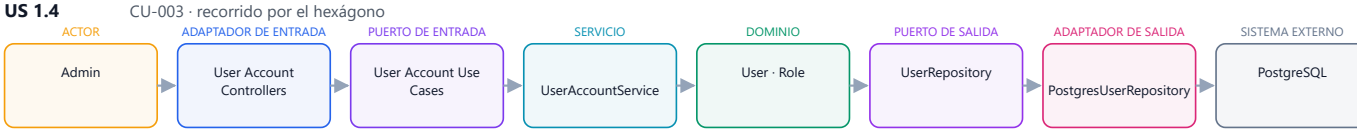
US 1.2 · Link an external authenticator application using a QR…
CU-001 · Iniciar sesión 2FA



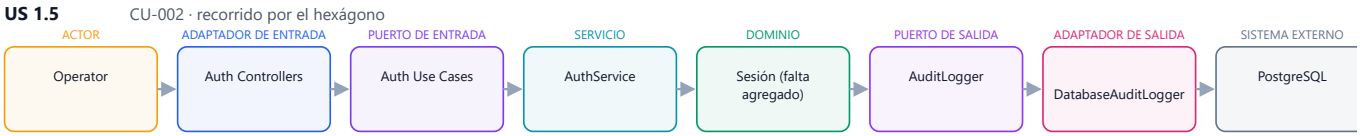
US 1.3 · Enter the 6-digit temporary code from my authenticator app after my…
CU-001 · Iniciar sesión 2FA



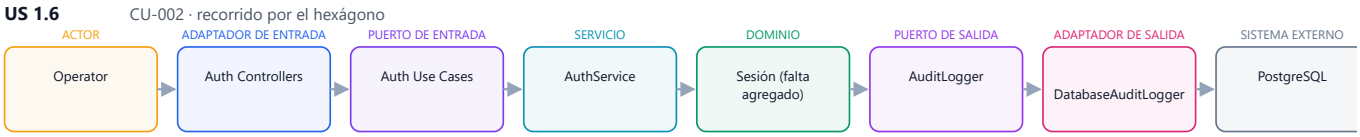
US 1.4 · Access to modules, views, and actions to depend exclusively on each account's role…
CU-003 · Administrar cuentas



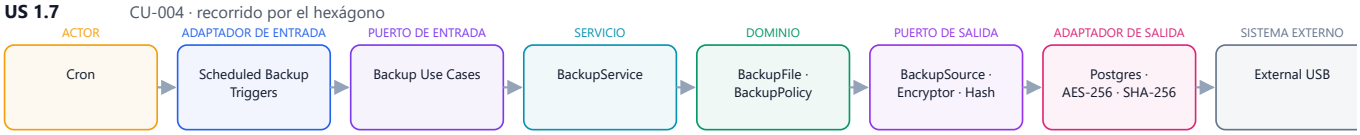
US 1.5 · My account to maintain only one active session at a…
CU-002 · Controlar sesión



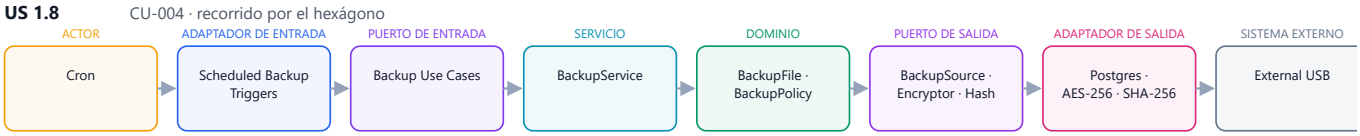
US 1.6 · My session to automatically close after a period of…
CU-002 · Controlar sesión



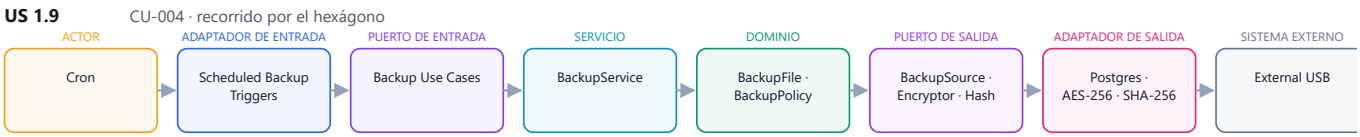
US 1.7 · The system to automatically generate an encrypted differential backup at the end of each…
CU-004 · Respaldo programado



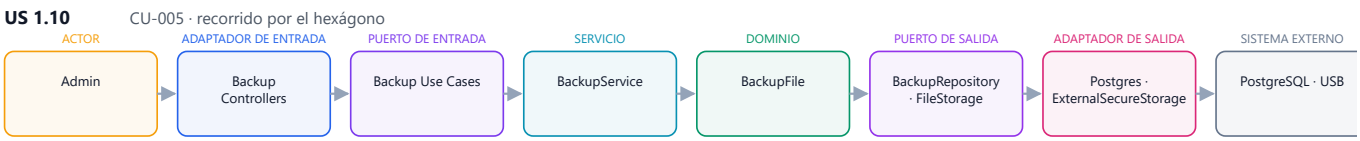
US 1.8 · The system to automatically rotate and purge backup versions according to a retention…
CU-004 · Respaldo programado



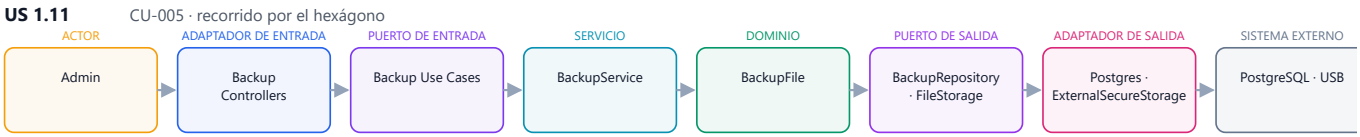
US 1.9 · Receive an email notification when automated backup…
CU-004 · Respaldo programado



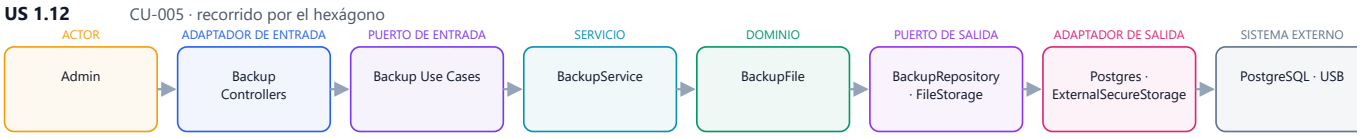
US 1.10 · Consult the inventory of available backup…
CU-005 · Inventario de respaldos



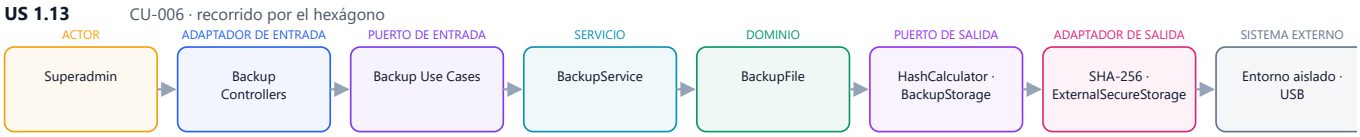
US 1.11 · Filter the backup inventory by various…
CU-005 · Inventario de respaldos



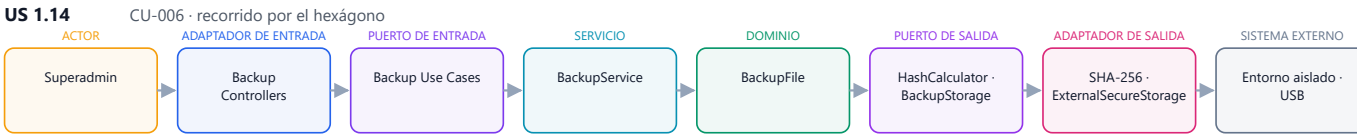
US 1.12 · Directly download a stored backup…
CU-005 · Inventario de respaldos



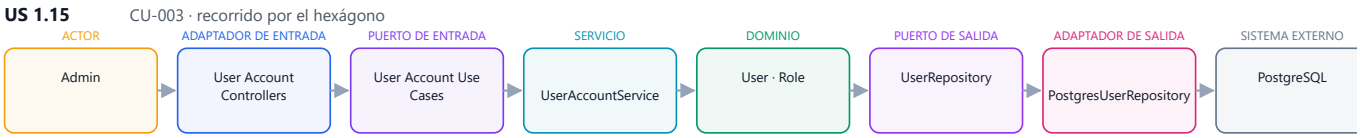
US 1.13 · The system to automatically test a backup in an isolated environment before restoring…
CU-006 · Restaurar respaldo



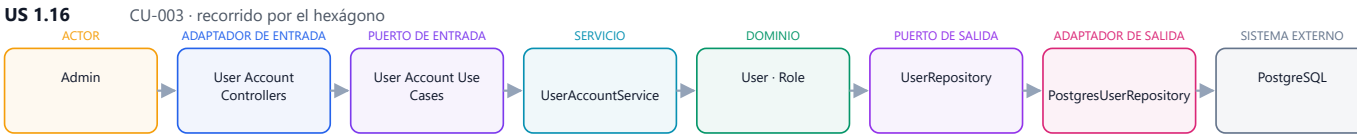
US 1.14 · Explicitly confirm restoration of a verified backup over the main…
CU-006 · Restaurar respaldo



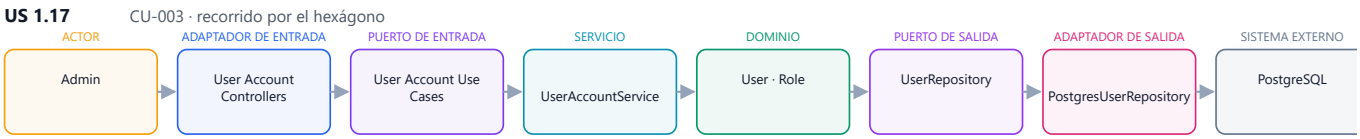
US 1.15 · Consult a paginated list of user…
CU-003 · Administrar cuentas



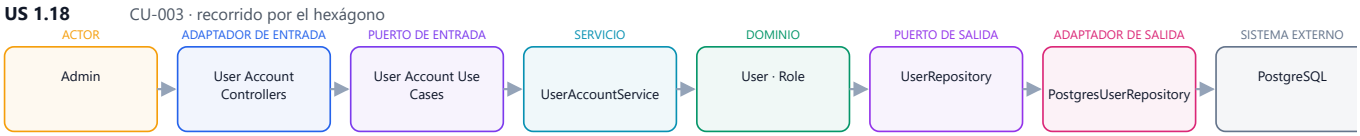
US 1.16 · Filter and search user…
CU-003 · Administrar cuentas



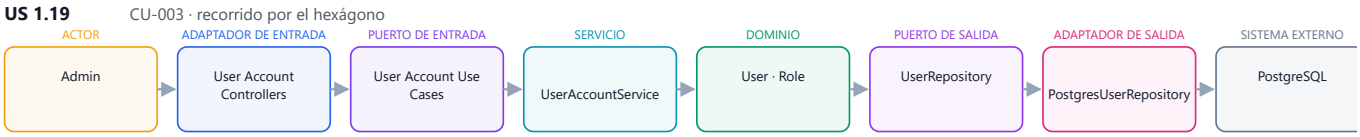
US 1.17 · Register new user…
CU-003 · Administrar cuentas



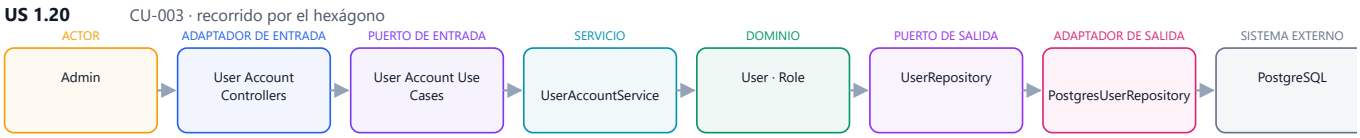
US 1.18 · Modify user account...
CU-003 · Administrar cuentas



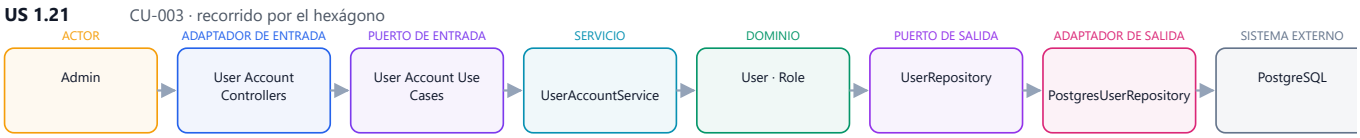
US 1.19 · Update a user account...
CU-003 · Administrar cuentas



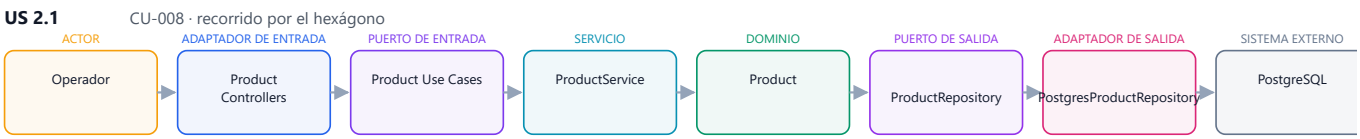
US 1.20 · Deactivate user accounts without deleting...
CU-003 · Administrar cuentas



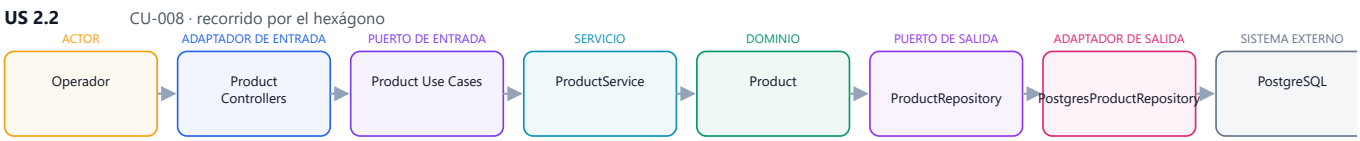
US 1.21 · Reactivate previously deactivated user...
CU-003 · Administrar cuentas



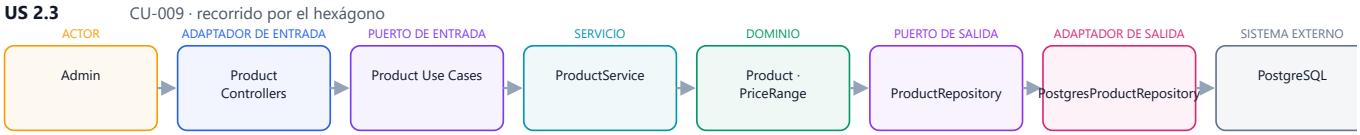
US 2.1 · Consult the product...
CU-008 · Consultar catálogo



US 2.2 · Filter products in the...
CU-008 · Consultar catálogo

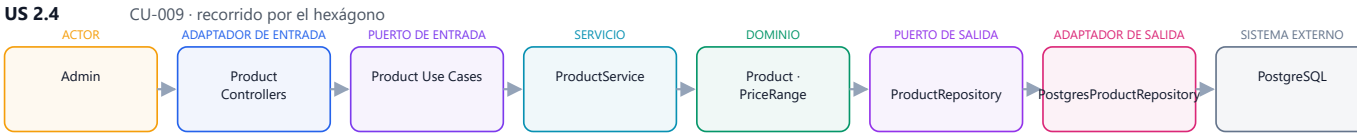


US 2.3 · Register new products in the central...
CU-009 · Administrar producto



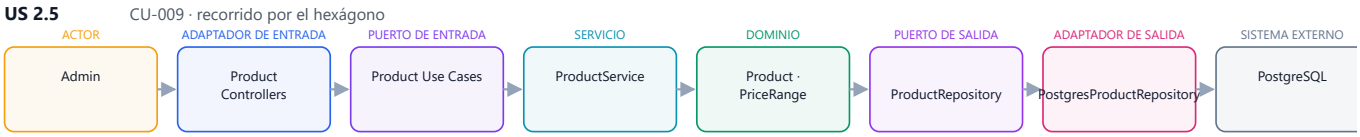
US 2.4 · Modify the description and unit of measure of a...

CU-009 · Administrar producto



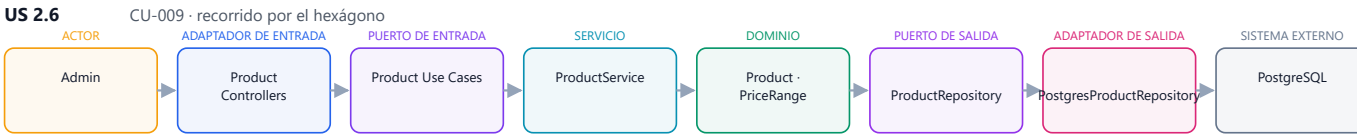
US 2.5 · Deactivate products without deleting...

CU-009 · Administrar producto



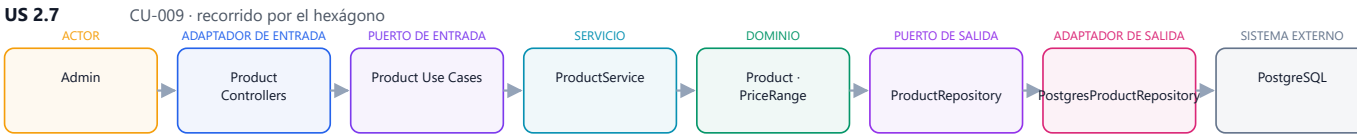
US 2.6 · Reactivate previously deactivated...

CU-009 · Administrar producto



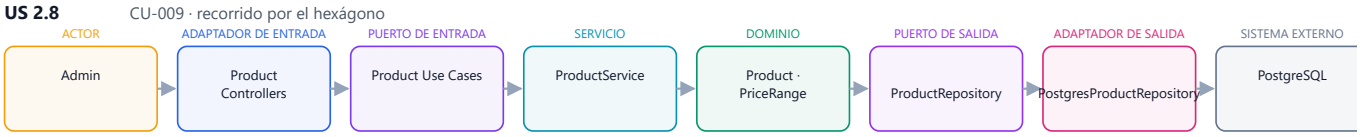
US 2.7 · Provide my contact details and required products and quantities to the web portal virtual...

CU-009 · Administrar producto



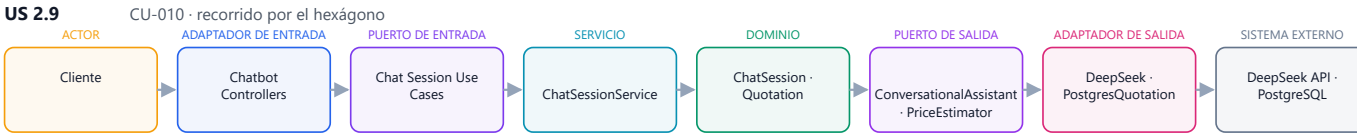
US 2.8 · Download my quote as a PDF document directly from the virtual...

CU-009 · Administrar producto

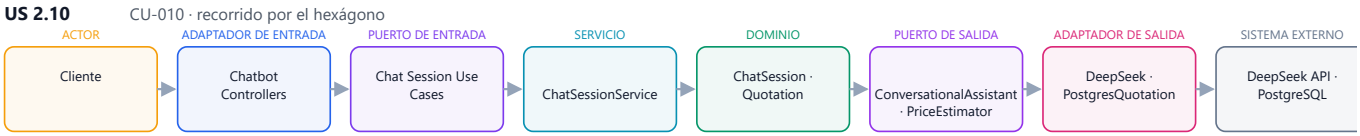


US 2.9 · Receive my quote as a PDF document by...

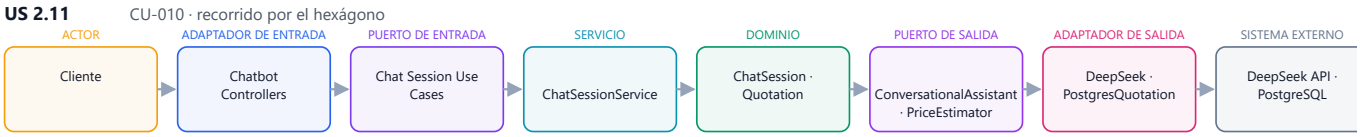
CU-010 · Cotización por asistente



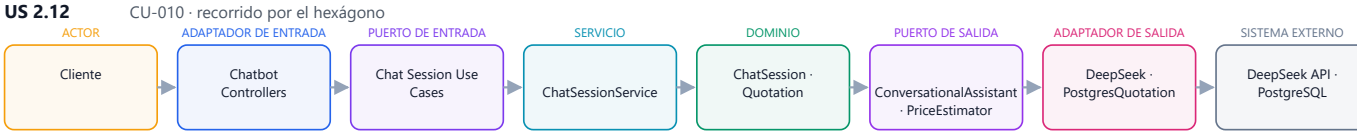
US 2.10 · Each quote generated by the virtual assistant to be automatically registered in the tracking...
CU-010 · Cotización por asistente



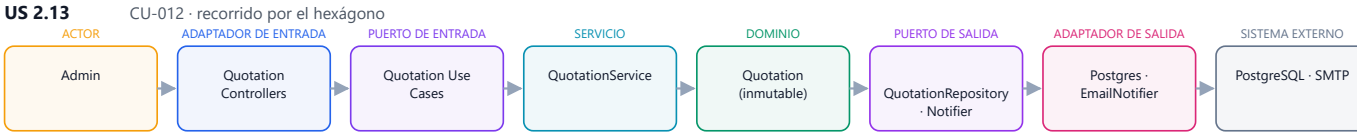
US 2.11 · The system to calculate and display a suggested dispatch date on...
CU-010 · Cotización por asistente



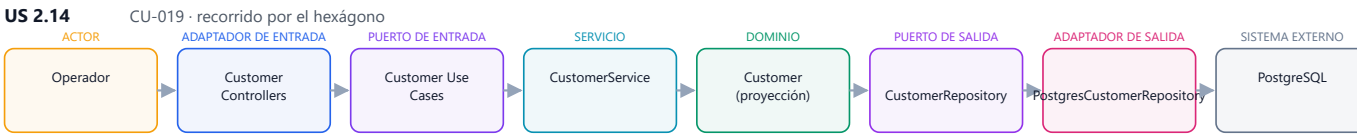
US 2.12 · Confirm the suggested dispatch date on a preliminary...
CU-010 · Cotización por asistente



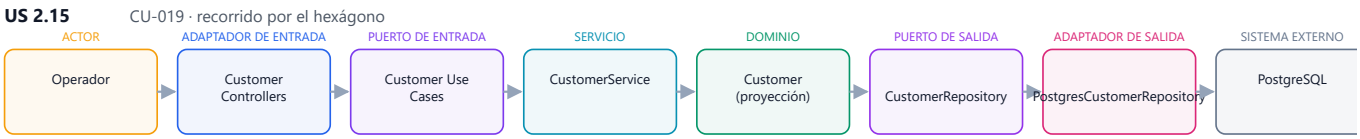
US 2.13 · Adjust the suggested dispatch date on a preliminary quote, or enter it manually when no...
CU-012 · Aprobar cotización



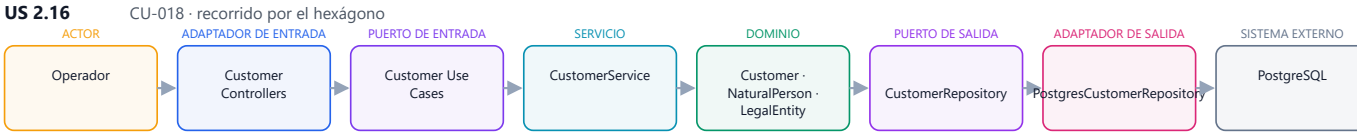
US 2.14 · Consult a paginated list of...
CU-019 · Ficha 360



US 2.15 · Filter and search...
CU-019 · Ficha 360

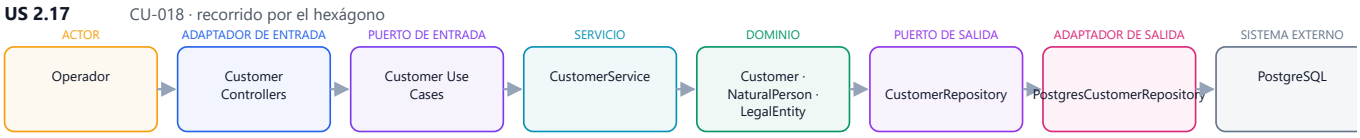


US 2.16 · Register new...
CU-018 · Gestionar clientes



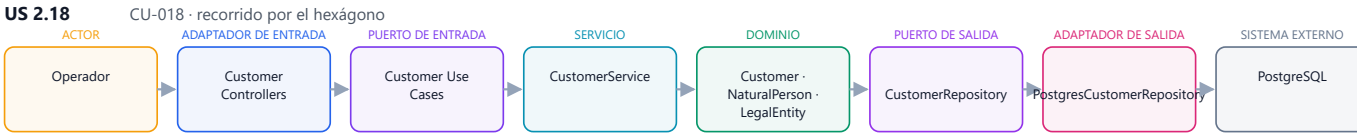
US 2.17 · Modify customer…

CU-018 · Gestionar clientes



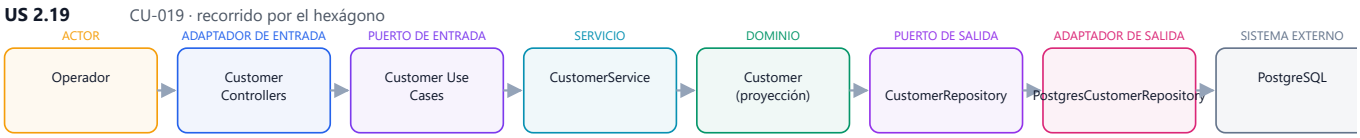
US 2.18 · Deactivate customers without deleting…

CU-018 · Gestionar clientes



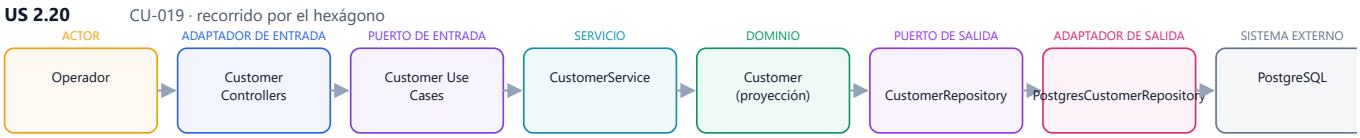
US 2.19 · Consult a unified sheet of a customer, identified by NIT or document…

CU-019 · Ficha 360



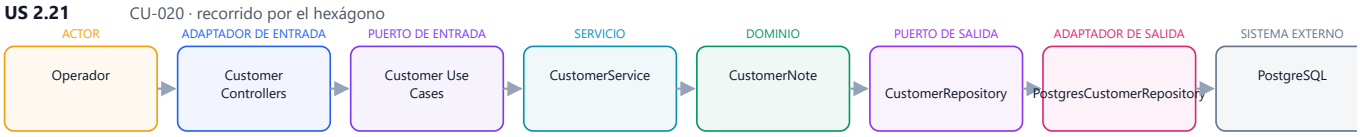
US 2.20 · Consult the chronological history of quotes and work orders for a…

CU-019 · Ficha 360



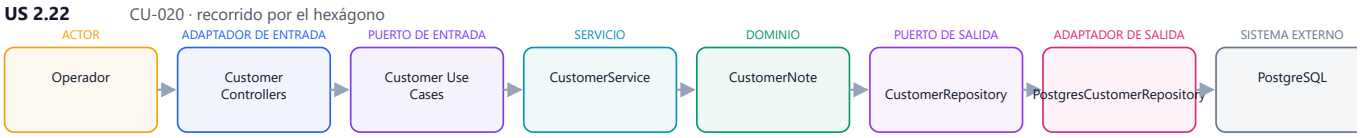
US 2.21 · Consult internal commercial notes on a customer's profile…

CU-020 · Notas comerciales

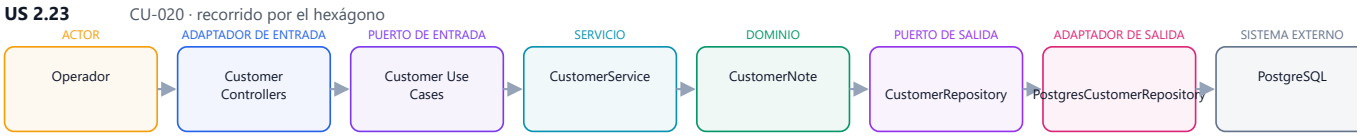


US 2.22 · Register an internal commercial note on a customer's profile…

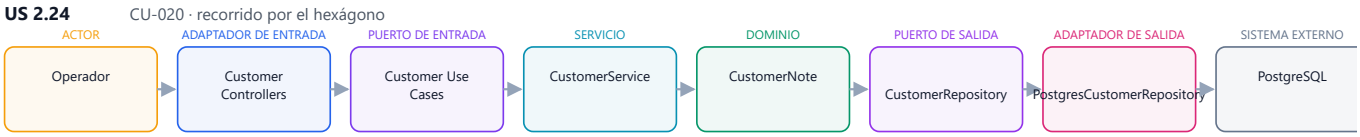
CU-020 · Notas comerciales



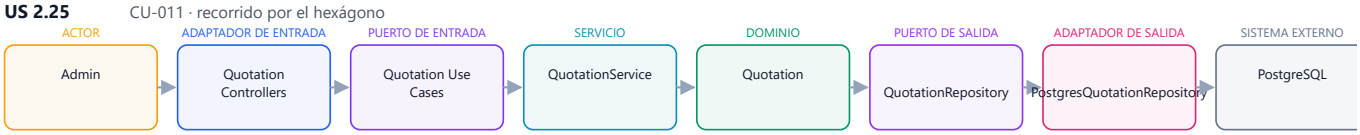
US 2.23 · Modify an internal commercial note that l…
CU-020 · Notas comerciales



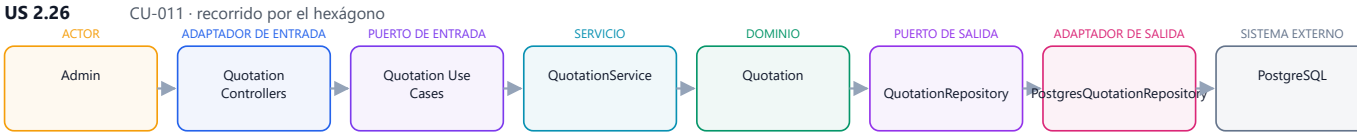
US 2.24 · Delete an internal commercial note from a customer's profile…
CU-020 · Notas comerciales



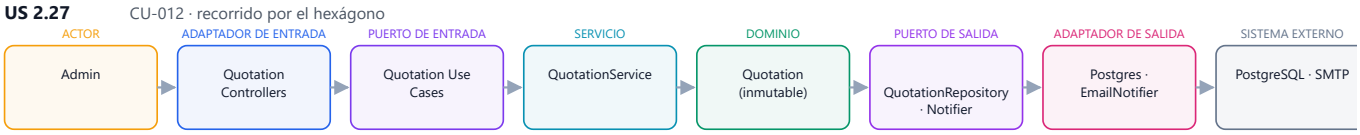
US 2.25 · Consult a paginated list of preliminary quotes generated by the virtual…
CU-011 · Consultar cotizaciones



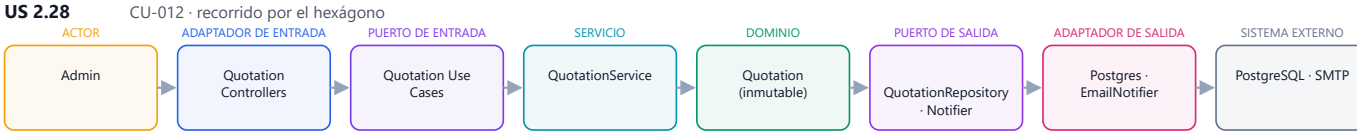
US 2.26 · Filter preliminary…
CU-011 · Consultar cotizaciones



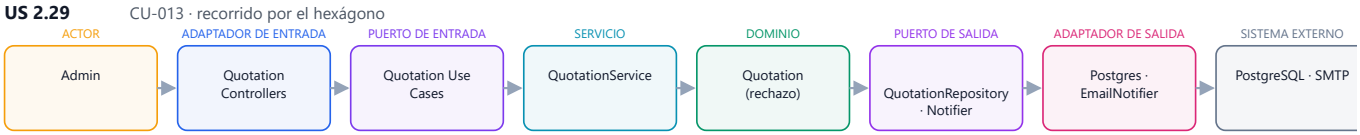
US 2.27 · Approve a preliminary…
CU-012 · Aprobar cotización



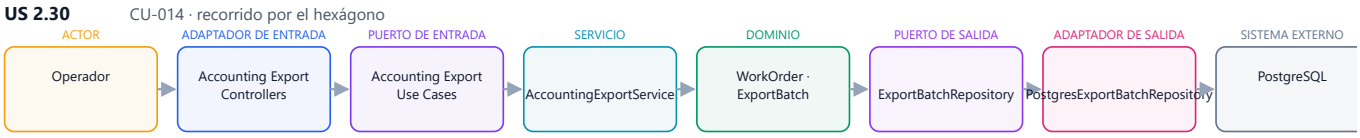
US 2.28 · Modify a preliminary…
CU-012 · Aprobar cotización



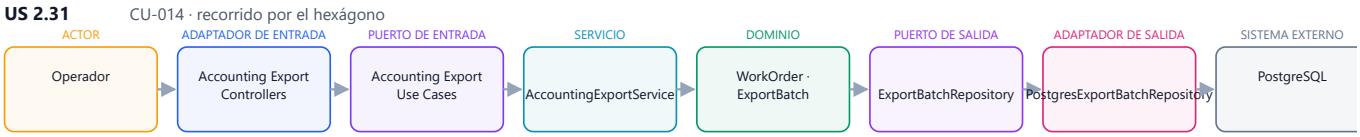
US 2.29 · Reject a preliminary quote by stating a…
CU-013 · Rechazar cotización



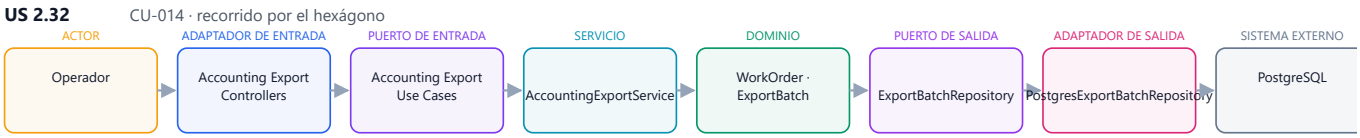
US 2.30 · Approved commercial orders to automatically enter a processing…
CU-014 · Cola de órdenes



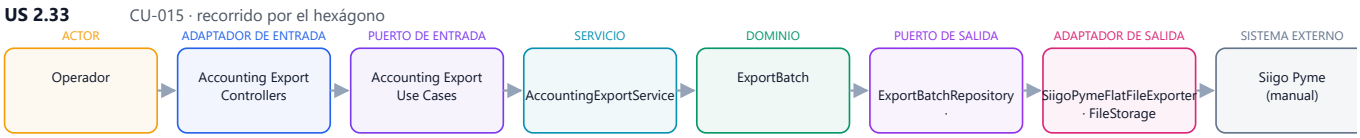
US 2.31 · Consult the queue of approved…
CU-014 · Cola de órdenes



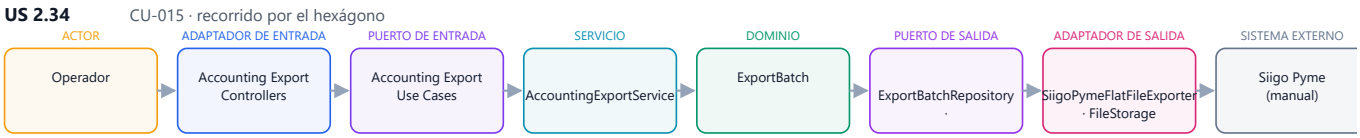
US 2.32 · Filter orders in the…
CU-014 · Cola de órdenes



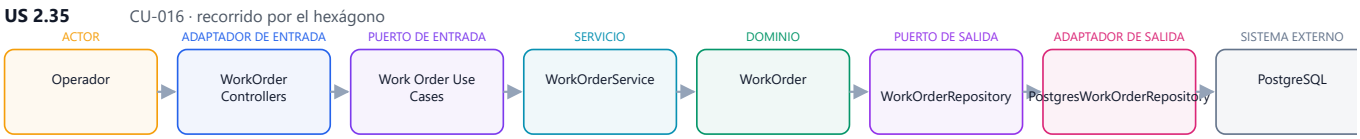
US 2.33 · Select multiple orders from the processing…
CU-015 · Lote para Siigo



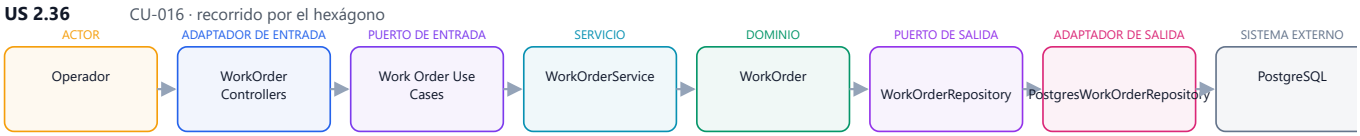
US 2.34 · Export selected orders into a single flat file for Siigo…
CU-015 · Lote para Siigo



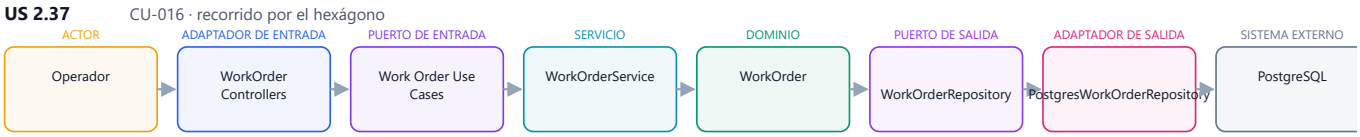
US 2.35 · Consult a paginated list of work…
CU-016 · Registrar O.T.



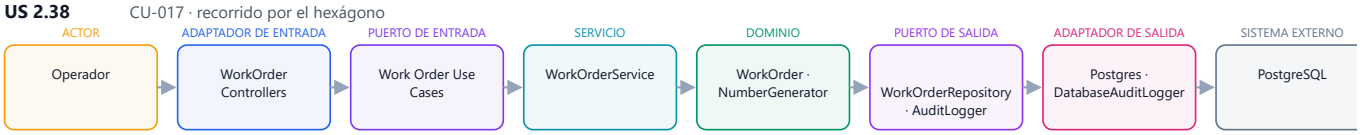
US 2.36 · Filter work…
CU-016 · Registrar O.T.



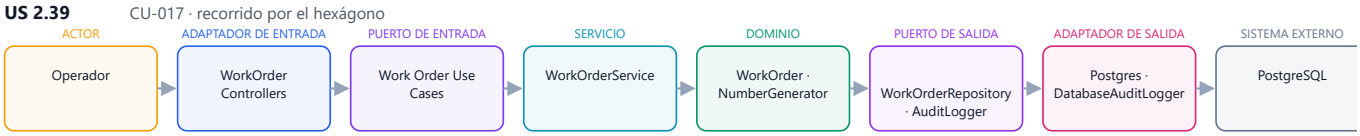
US 2.37 · Manually register a new work…
CU-016 · Registrar O.T.



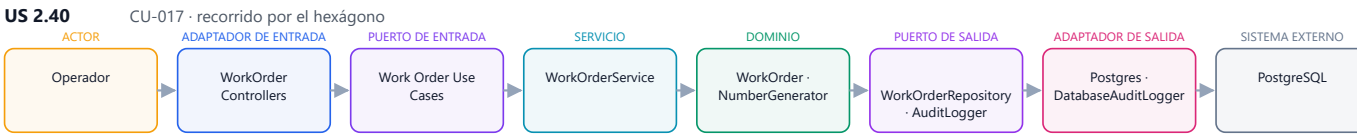
US 2.38 · Modify an existing work…
CU-017 · Modificar O.T.



US 2.39 · Annul a work order stating a…
CU-017 · Modificar O.T.



US 2.40 · Update work order status following its established…
CU-017 · Modificar O.T.



15. El repositorio `domain-driven-hexagon`: por qué parece que todo es DDD

El curso cita `sairyss/domain-driven-hexagon` (MIT, unos 15.000 votos en GitHub) y su README lo dice en la primera línea de la sección de arquitectura:

> «This is an attempt to **combine multiple architectural patterns and styles together**, such as: Domain-Driven Design (DDD), Hexagonal (Ports and Adapters) Architecture, Secure by Design, Clean Architecture, Onion Architecture, SOLID Principles, Software Design Patterns, and many others.»

Es decir: **ese repositorio no es «DDD»; es una combinación declarada**. Su diagrama no contradice nada de lo anterior: muestra el estilo hexagonal (puertos y adaptadores) aplicado **dentro** de un diseño guiado por el

dominio. Las dos cosas conviven porque responden a preguntas distintas — y eso es exactamente lo que confundía al grupo.

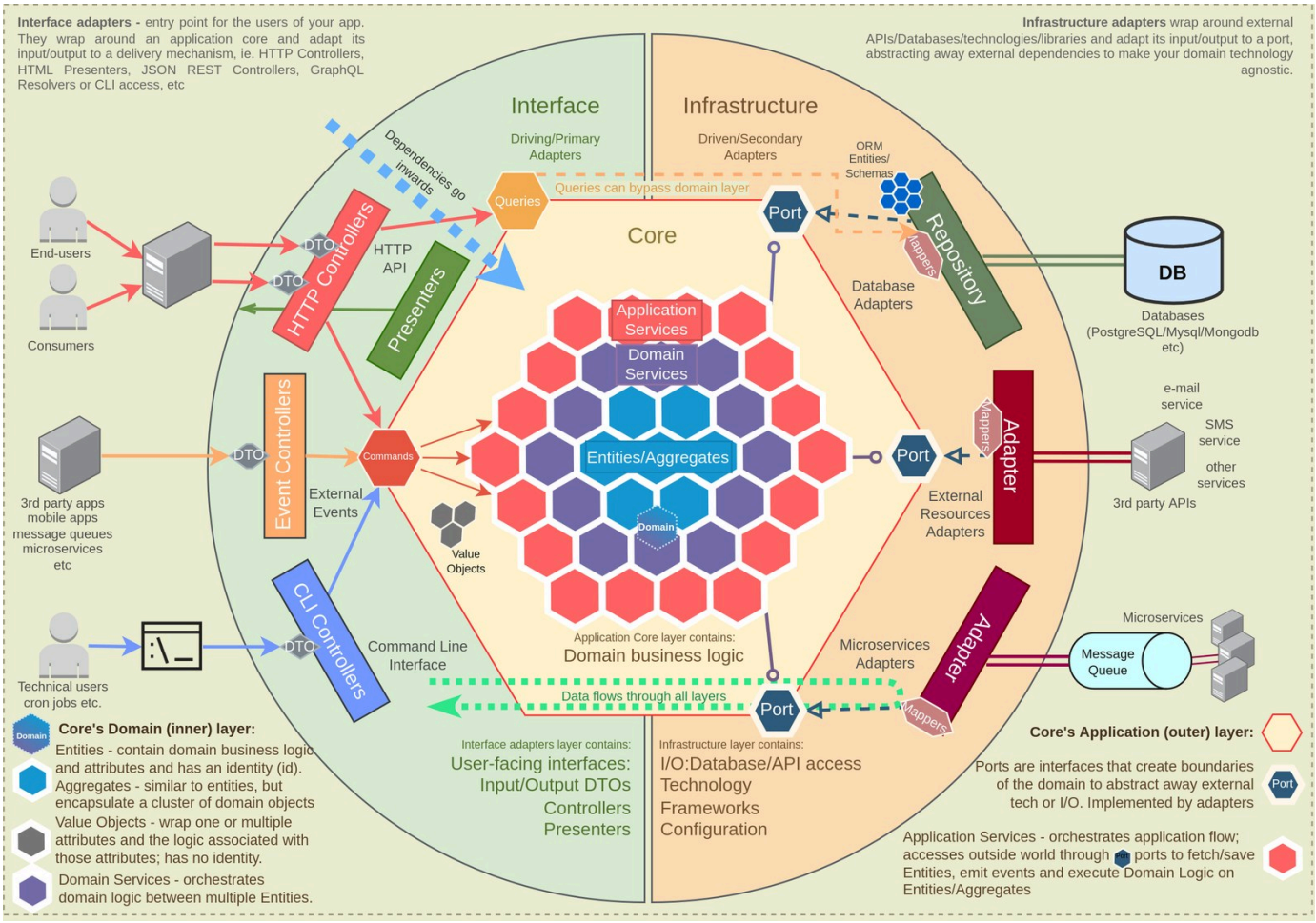


Diagrama de `sairyss/domain-driven-hexagon` (MIT), basado a su vez en el de `hgraca/explicit-architecture-php`. Se reproduce con atribución.

15.1 Qué muestra y cómo se traduce al diagrama de IMSI

En la imagen	Qué es	En el diagrama de la entrega
Interface adapters (HTTP, CLI, event controllers, presenters)	adaptadores de entrada; reciben DTO	**Adaptadores de entrada**: Chatbot, Auth, Quotation, Backup Controllers, Celery Beat
Commands / Queries	objetos de entrada del caso de uso (CQRS)	**Puertos de entrada**: `Quotation Use Cases`, etc.
Application Services	orquestan el caso de uso	**Servicios de aplicación**: `QuotationService`, `BackupService`
Domain: Entities / Aggregates, Value Objects, Domain Services	el negocio	**Dominio**: `Quotation`, `WorkOrder`, `Money`, `PriceRange`, `BackupPolicy`
Ports	contratos hacia el exterior	**Puertos de salida**: `QuotationRepository`, `Notifier`, `Encryptor`

En la imagen	Qué es	En el diagrama de la entrega
Adapters (Repository, external resources, microservices)	implementaciones	**Adaptadores de salida**: `PostgresQuotationRepository`, `EmailNotifier`, `SiigoPymeFlatFileExporter`
DB, external services, message queue	fuera del sistema	**Sistemas externos**: PostgreSQL, SMTP, DeepSeek, Siigo, USB

La dirección de las flechas es la misma que ya está en el documento: hacia adentro. La imagen lo dibuja con más adornos (DVO, presenters, CQRS), pero la regla es idéntica a la del capítulo 3.

15.2 Qué toma IMSI y qué deja fuera

Pieza del repositorio	¿En IMSI?	Por qué
Puertos y adaptadores	**sí**	es la razón del estilo
Agregados, entidades, value objects, políticas	**sí**	el dominio tiene reglas reales (inmutabilidad, rangos, estados)
Repositorios con mapper fila ↔ agregado	**sí**	`Mapper (Domain Entity ⇔ SQL Row)` está en el diagrama
Eventos de dominio	**sí**	`QuotationApproved` dispara cola, PDF y correo
Commands y Queries separados (CQRS)	**no**	no lo pide ningún requisito; añadiría dos archivos por caso de uso
Presenters	**no**	el frontend es Next.js; el presentador no vive en el backend
Cola de mensajes entre módulos	**no**	los módulos se llaman en proceso; solo Siigo sale por archivo
Microservicios	**no**	hay un despliegue

La lección práctica: ese repositorio es un **catálogo de opciones**, no un molde que haya que copiar entero. Tomar CQRS o los presenters sin necesidad es justamente el sobrecoste que el capítulo 4.3 advierte.

15.3 La carpeta, comparada

El repositorio organiza cada módulo en vertical:

```
src/modules/<modulo>/
├── api/            # controladores, DTOs de entrada y salida
├── application/    # comandos, queries, servicios de aplicación, puertos
├── domain/         # entidades, agregados, value objects, eventos
└── infrastructure/ # repositorios, ORM, adaptadores externos
```

Es la misma idea de las carpetas del capítulo 8, con otro nombre:

```
mi_modulo/
├── domain/         # ← domain
├── application/    # ← application
```

```
├── adapters/inbound/   # ← api
└── adapters/outbound/  # ← infrastructure
```

La diferencia de fondo no es de estilo sino de tecnología: el repositorio usa NestJS y TypeScript; IMSI usa Django y Python. Los nombres cambian; la dirección de las dependencias, no.

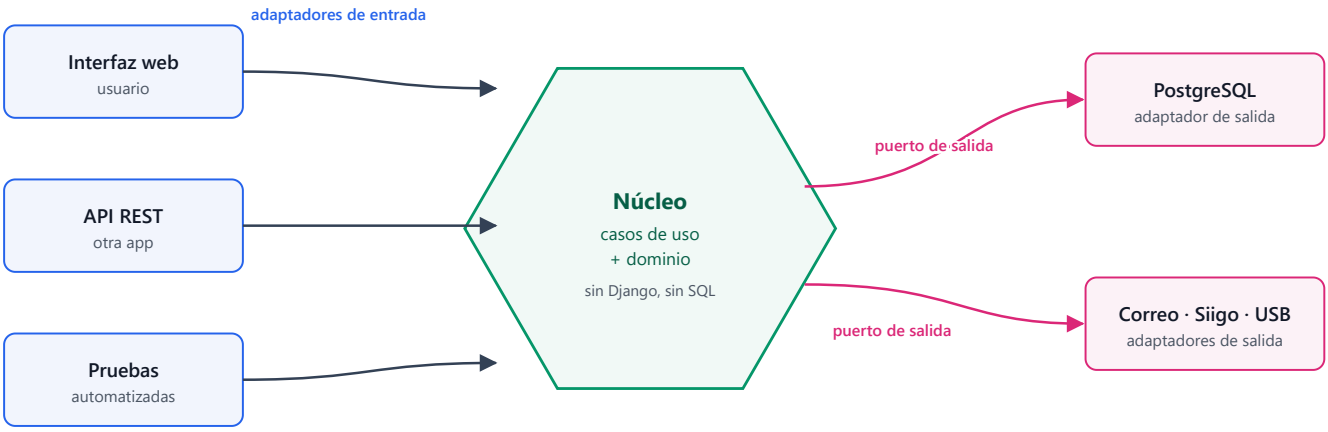
Y una idea que sí vale la pena copiar: ese repositorio mantiene una **gráfica de dependencias** del código (`assets/dependency-graph.svg`, generada con madge) para vigilar que los módulos no se importen entre sí. En IMSI el equivalente es un test de arquitectura con `import-linter` o un script que revise imports: es la única forma de que la regla del capítulo 3 no se degrade sola.

15.4 El diagrama mínimo, y por qué el otro parece complicado

No, no está complejizado a propósito. Está **superpuesto**: el patrón es una sola decisión y el dibujo famoso le suma todo lo que *puede* ir dentro.

Alistair Cockburn, el autor, lo resumió así cuando le preguntaron qué hay dentro del hexágono: *«I don't care – not my business»*, y describió el patrón completo como una única decisión: **«wrap your app in an API and put tests around it»**.

Ese es el diagrama mínimo, y es el que corresponde a IMSI:



Las dependencias del código apuntan siempre hacia el núcleo: el adaptador conoce el puerto, el núcleo no conoce el adaptador.

Cuatro cosas, ni una más:

1. Un **núcleo** con los casos de uso y el dominio, sin Django, sin SQL.
2. **Puertos de entrada** por donde lo controlan: UI, API, pruebas.
3. **Puertos de salida** hacia lo que él controla: base de datos, correo, Siigo, USB.
4. **Adaptadores** a cada lado; varios por puerto, si hace falta.

El resto del diagrama de `domain-driven-hexagon` —Commands, Queries, Presenters, colas, microservicios— son **decisiones opcionales** que ese repositorio tomó. Cada caja extra responde «esto también podría estar aquí», no «esto tienes que tenerlo».

15.5 Las tres advertencias del artículo de HappyCoders

El artículo de HappyCoders (Sven Woltmann, actualizado en septiembre de 2026) es la explicación más limpia que hay del patrón, y tres de sus puntos aplican directo al proyecto:

Punto del artículo	Qué significa para IMSI
El dilema del mapeo. Si el núcleo no puede conocer el ORM, la entidad de dominio y la fila de la base son dos clases distintas	Es exactamente el `Mapper (Domain Entity ↔ SQL Row)` del diagrama. La estrategia recomendada es el mapeo en dos direcciones
Pruebas con dobles. Los puertos de salida se sustituyen por dobles en las pruebas	Es el `InMemoryQuotationRepository` del capítulo 8.8: probar reglas sin PostgreSQL, requisito de NFR-05-02
No vale para todo. El artículo advierte que en CRUD simple el esfuerzo no se amortiza	Catálogo y clientes son CRUD: ahí el hexágono es ceremonia. Se acepta por uniformidad, sabiendo que no aporta

Y una aclaración que cierra la discusión de estilos: hexagonal, **clean** y **onion** son casi lo mismo. Cambian los nombres de las capas, no la regla. El artículo lo demuestra superponiendo los tres diagramas. Discutir cuál de los tres «somos» es discutir vocabulario; lo que se evalúa es si las dependencias apuntan al núcleo.

15.6 La frase que se puede decir en la sustentación

> *«Nuestro diagrama es el mínimo del patrón: un núcleo, puertos y adaptadores. Lo que lo hace parecer grande es que cada caja del diagrama de referencia es una opción que ese proyecto tomó; nosotros tomamos las que los requisitos pedían y dejamos fuera CQRS, presenters y colas, porque no aportan a nuestras metas.»*

Eso es mejor que defender el dibujo completo: demuestra que se entendió el patrón y que se decidió qué no usar.

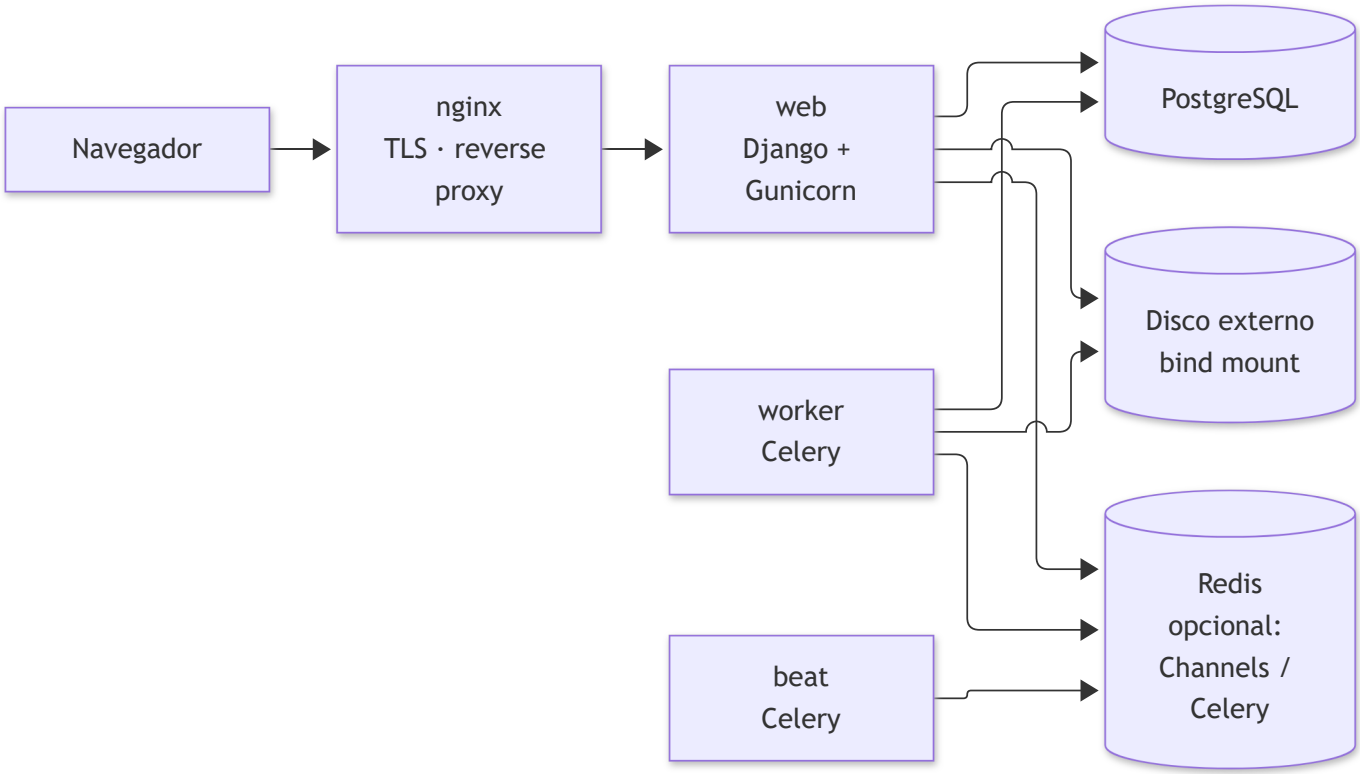
---

16. Contenedores y CI: ¿caben en esta arquitectura?

Sí, y ninguno de los dos cambia el estilo. Hexagonal decide **cómo se organizan las dependencias dentro del código**; Docker decide **cómo se empaqueta** y Actions decide **cómo se verifica**. Son dimensiones distintas: se puede tener hexagonal con o sin contenedores, y monolitos con o sin CI.

16.1 Docker: sí, y sin tocar el diagrama

El diseño ya es un **monolito modular**: un proceso Django + workers Celery. Los contenedores no lo convierten en microservicios; empaquetan ese mismo monolito en piezas que comparten una base de datos:



Lo que gana el proyecto:

Beneficio	Por qué importa aquí
Entorno reproducible	Python 3.12, PostgreSQL 16 y las dependencias exactas, igual en el portátil de cada uno y en el servidor de la empresa
Paridad con CI	Actions construye la misma imagen que se despliega: si pasa en CI, corre igual
Despliegue y rollback	cambiar de versión es cambiar la etiqueta de la imagen; volver atrás es usar la anterior
Aislamiento	nginx, Django y la base no comparten sistema operativo ni versiones
Sin orquestador	un solo servidor: `docker compose` basta; Kubernetes sería sobrecoste

Lo que hay que cuidar:

1. **El disco externo de los respaldos** se monta con `bind`, no con volumen anónimo: si el contenedor se recrea y el montaje se pierde, el respaldo se escribe en el contenedor y desaparece al reiniciarlo. Es el mismo fallo que ya está documentado en otro stack de este workspace.
2. **PostgreSQL con volumen nombrado** y su propio respaldo: el volumen no es un respaldo.
3. **Secretos por variables de entorno o Docker secrets**, nunca en la imagen ni en el repositorio.
4. **Un solo Dockerfile multi-etapa** para web, worker y beat: tres procesos, una imagen. Así no hay deriva entre lo que ejecuta la web y lo que ejecuta el worker.
5. **Healthchecks** en base y web; `depends_on` con condición de salud, no solo de arranque.

Y una advertencia de alcance: contenedores **no** son microservicios. Si en la sustentación alguien confunde las dos cosas, la respuesta es la misma del capítulo 2.3: hay un despliegue lógico, con varios procesos; los módulos siguen llamándose en proceso.

16.2 GitHub Actions: lo que el requisito ya pide

NFR-05-02 exige pipeline en GitHub Actions, 100 % de pruebas en verde, cobertura mínima del 70 % y datos sintéticos, con bloqueo de despliegue si algo falla. O sea: no es una mejora opcional, es una obligación del análisis.

Un pipeline mínimo que cumple:

```yaml
name: ci
on: [push, pull_request]
jobs:
  calidad:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_PASSWORD: postgres, POSTGRES_DB: imsi_test }
        ports: ["5432:5432"]
        options: >-
          --health-cmd pg_isready --health-interval 10s
          --health-timeout 5s --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: ruff check .
      - run: python manage.py makemigrations --check --dry-run
      - run: pytest --cov --cov-fail-under=70
      - run: lint-imports          # import-linter: el dominio no importa Django
      - run: docker build -t imsi:${{ github.sha }} .
```

Cinco controles, cada uno atado a algo real:

Paso	Qué previene
`ruff`	errores de estilo y bugs obvios antes de revisar
`makemigrations --check`	modelos cambiados sin migración: el clásico fallo al desplegar
`pytest --cov-fail-under=70`	cumple NFR-05-02 y frena la caída de cobertura
`lint-imports`	**la regla del capítulo 3**: nadie importa `django` desde `domain/`
`docker build`	la imagen que se despliega se construye de verdad

16.3 Lo que Actions **no** resuelve

Hay que decirlo claro, porque es la expectativa equivocada:

- **No evita conflictos de merge.** Un conflicto lo produce tocar las mismas líneas. Lo que evita conflictos es trabajar en ramas cortas, integrando seguido, y hacer `rebase` antes de abrir el PR. Los Actions

detectan el error *después* del merge, no antes.

- **No detecta un merge semántico mal hecho** si las pruebas no cubren ese caso. Con cobertura del 70 %, un tercio del código no está protegido.
- **No sustituye la revisión.** Un PR con checks en verde y sin revisión sigue siendo un PR sin revisión.

La combinación que sí funciona, y es barata: ramas de un día, PR con dos aprobaciones, checks obligatorios en `main` (branch protection), y migraciones revisadas a mano porque los tests no las validan contra datos reales.

16.4 ¿Se pueden escribir los Actions con un asistente de IA?

Sí: un workflow es YAML, y un asistente lo escribe y lo corrige como cualquier otro archivo. Pero el valor no está en quién lo teclea, sino en tres cosas que el asistente no adivina solo:

1. **Qué debe correr**: la lista de arriba sale de NFR-05-02 y de la regla de dependencias, no de la plantilla por defecto.
2. **Qué debe bloquear**: la protección de rama se configura en GitHub, no en el YAML; sin ella los checks son decorativos.
3. **Qué datos usa**: pruebas con datos sintéticos, nunca con la base real de la empresa.

Con eso claro, pedirle el workflow a un asistente es razonable: se revisa el YAML, se ejecuta en un PR de prueba y se ajusta. Lo que no es razonable es aceptar un pipeline generado sin leerlo: un `pytest -k ""` o un `|| true` al final convierten la puerta en un adorno.